

Sri Lanka Institute of Information Technology



Data Warehousing and Business Intelligence IT3021 B.Sc. (Hons) in Information Technology DATA SCIENCE

Student Name: KATIPEARACHCHI M C

Registration Number: IT23234420

Group: Y3S2 W.E 1.1

Date: 30.03.2026

Table of Contents

Declaration.....	4
Acknowledgement.....	5
Task 1: Data set selection.....	6
1.0 Introduction to the Dataset.....	6
1.1 Data Set Selection.....	7
1.2 Entity Relationship Diagram.....	8
Task 2: Preparation of Data Sources.....	10
2.0 Introduction.....	10
2.1 Data Descriptions.....	12
Task 3: High Level Design Architecture.....	15
Task 4: Data Warehouse design and Development.....	17
4.0 Introduction.....	17
4.1 Conceptual Data model.....	17
4.2 Dimensional Model Design.....	18
4.3 Fact Table Description.....	21
4.4 Dimension Table Description.....	22
4.4.1 DimDate.....	22
4.4.2 DimOffense.....	22
4.4.3 DimLocation.....	22
4.4.4 DimPoliceArea.....	22
4.5 Keys and Relationships.....	24
4.6 Design Assumptions.....	25
4.7 Validation of the Dimensional Model.....	25
Task 5: ETL Development.....	26
5.0 Overview of the ETL process.....	26
5.1 Data Profiling.....	27
5.2 Use of Multiple Source Types and ETL Extraction.....	29
5.3 ETL Transformations.....	31
5.3.1 Data Conversion.....	31
5.3.2 Lookup.....	33

5.3.3 Derived Column.....	35
5.3.4 Error Handling.....	38
5.4 Staging Table Loads.....	41
5.5 Dimension Loads.....	44
5.6 Slowly Changing Dimension Implementation – DimPoliceArea.....	47
5.7 Fact Table Load – FactCrimeIncident.....	48
5.8 Yearly Summary Aggregate Load.....	49
Task 6: ETL development – Accumulating fact tables.....	51
6.0 Extending the fact table.....	51
6.1 Initial fact load.....	51
6.2 Separate completion dataset.....	52
6.3 Separate SSIS package.....	53
6.4 Update logic.....	54
6.5 Package configuration.....	54
6.6 Validation.....	58

Declaration

I declare that this project report, or any part of it, is not copied from any document produced by any organization, university, or other institution, or from any previous student project at SLIIT. It has also not been copied from the internet or any other sources.

Student Details

Registration Number : IT23234420

Name in Full : KATIPEARACHCHI M.C

Batch : Y3S2 1.1 WEEKEND

Acknowledgement

I would like to express my sincere gratitude to the Lecturer-in-Charge, **Ms. Gaya Thamali Dissanayake**, for her continuous guidance, encouragement, and supervision throughout the successful completion of this assignment. Her valuable advice and support greatly contributed to the development of this work.

I would also like to extend my heartfelt thanks to the **lab instructors** for their assistance, patience, and support provided during the practical implementation of this assignment. Their guidance helped me to overcome many challenges encountered during the process.

Finally, I wish to convey my special appreciation to the **Sri Lanka Institute of Information Technology (SLIIT)** for providing the necessary resources, facilities, and a supportive learning environment to successfully complete this assignment.

Task 1: Data set selection

1.0 Introduction to the Dataset

The dataset selected for this project is **“Crimes - 2001 to Present.”** For this assignment, a local snapshot of the dataset was used. The selected snapshot contains **759,161 records** and **22 attributes**, covering the period from **1 January 2023 to 1 January 2026**.

Each record represents a unique reported crime incident and includes operational details such as incident ID, case number, date, offence code, offence type, location, arrest status, domestic status, and police administrative area information. As the dataset is structured at event level, it is suitable for use as an **OLTP-style** source.

This dataset was selected to support the business scenario **Urban Public Safety Incident Response and Community Risk Intelligence**. It provides an appropriate basis for the design and implementation of a **Data Warehouse and Business Intelligence (DW/BI)** solution, including source preparation, dimensional modelling, ETL development, and analytical reporting.

1.1 Data Set Selection

Data Set Name: Crimes - 2001 to Present

Provided by : Chicago Police Department

Source link : https://data.cityofchicago.org/Public-Safety/Crimes-2001-to-Present/ijzp-q8t2/data_preview

This dataset is published by the **Chicago Police Department** through the **City of Chicago Open Data Portal**. It contains reported crime incidents that occurred within the City of Chicago from 2001 onward, excluding the most recent seven days. The data is extracted from the **Citizen Law Enforcement Analysis and Reporting (CLEAR)** system and is updated daily. The full dataset contains approximately **8.52 million records** and **22 attributes**.

The dataset is suitable for analysing crime patterns from multiple perspectives, including crime type, time, location, arrest status, and community-based trends. It supports the identification of long-term crime trends, high-risk areas, and temporal variations in criminal activity, making it suitable for academic and analytical purposes.

The dataset also preserves privacy while maintaining analytical value. Incident addresses are presented only at block level, and exact locations are not disclosed. As the records are based on preliminary police reports, some values may be revised during investigations. Nevertheless, the dataset remains a useful and realistic source for data analysis and visualisation in the field of public safety.

1.2 Entity Relationship Diagram

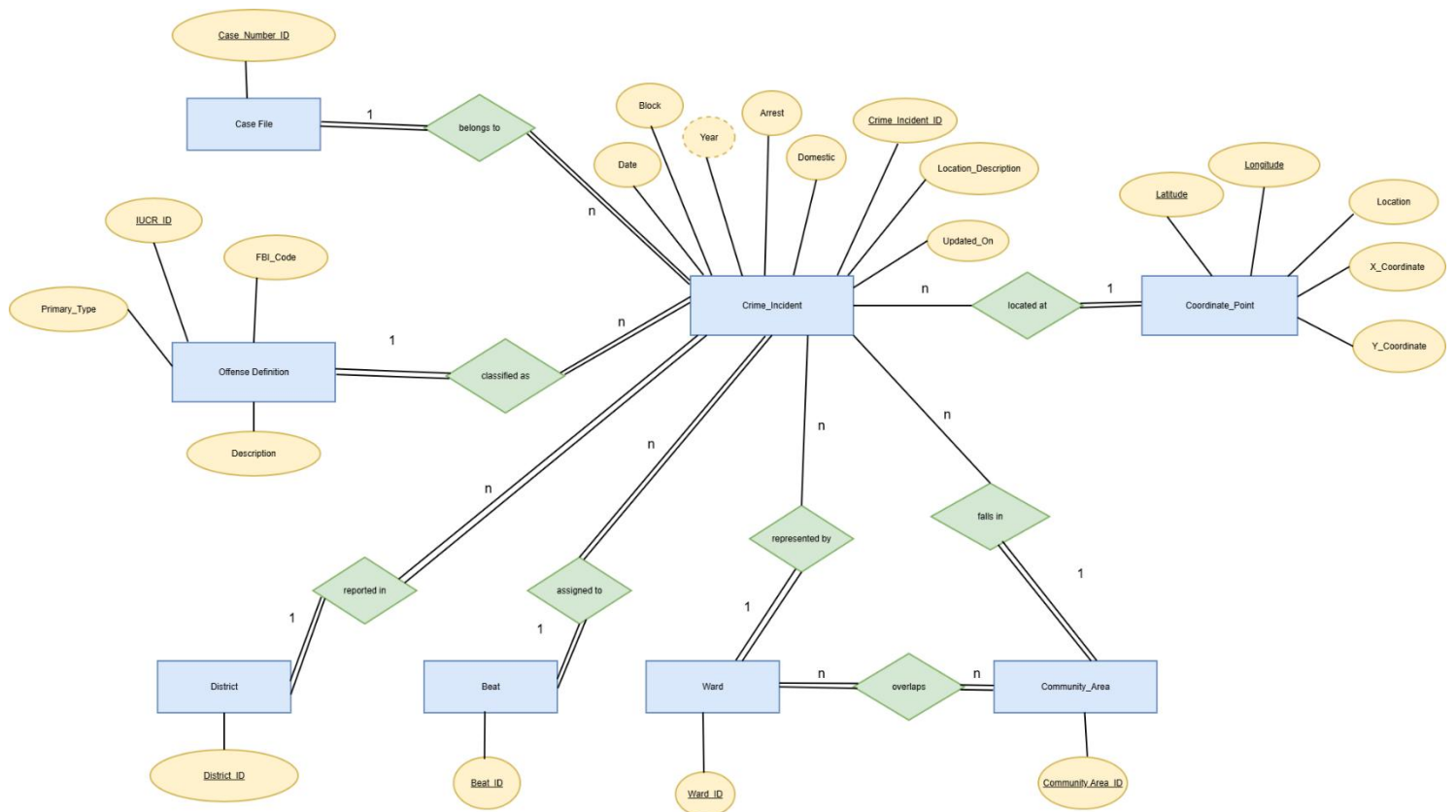


Figure 1: ER Diagram for the Operational Crime Incident Database

An Entity Relationship (ER) diagram was used to represent the prepared operational source structure prior to the development of the data warehouse. Within the source model, the central entity is Crime Incident, as the source grain is defined as one record per reported incident.

Around this core entity, the following related entities were identified:

- Case File
- Offense Definition
- Coordinate Point
- District
- Beat
- Ward
- Community Area

These entities were identified because they represent business values that recur across multiple incident records. For example, a Case Number may associate

multiple incidents with a single case file, the IUCR code identifies reusable offence definitions, and the Latitude–Longitude pair represents reusable coordinate points. Similarly, District, Beat, Ward, and Community Area are recurring geographic or policing-area codes associated with many crime incidents.

The main relationships in the prepared source model are as follows:

- Case File 1 : N Crime Incident
- Offense Definition 1 : N Crime Incident
- Coordinate Point 1 : N Crime Incident
- District 1 : N Crime Incident
- Beat 1 : N Crime Incident
- Ward 1 : N Crime Incident
- Community Area 1 : N Crime Incident
- Ward N : N Community Area

These relationships were derived from the selected source snapshot. A direct District 1 : N Beat hierarchy was intentionally not enforced, as the selected snapshot did not provide sufficient support for a clear and consistent one-directional relationship between these two codes.

Task 2: Preparation of Data Sources

2.0 Introduction

The original crime dataset was prepared as three separate source files to support a clearer and more structured source environment for ETL and dimensional modelling. Rather than relying on a single flat operational file, the data was reorganised into logically distinct sources representing incident-level data, offence reference data, and coordinate reference data.

The prepared source files are as follows:

- **crimes_original_reduced.csv**
- **offense_source.txt**
- **coordinate_source.xlsx**

These files represent three different source types:

- **CSV**
- **Text**
- **Excel**

The main incident-level source is stored as a **CSV** file, the offence lookup source is stored as a **tab-delimited text** file, and the coordinate lookup source is stored as an **Excel workbook**.

The data stored in each source is summarised below:

- **crimes_original_reduced.csv**
Stores the main incident-level transactional data. It includes attributes required to represent a crime event, such as incident ID, case number, date, block, arrest flag, domestic flag, policing-area codes, and offence code.
- **offense_source.txt**
Stores offence reference information keyed by **IUCR**. It includes attributes such as **Primary Type**, **Description**, and **FBI Code**, and functions as an offence definition lookup source.
- **coordinate_source.xlsx**
Stores reusable coordinate reference data keyed by **Latitude** and **Longitude**. It includes **X Coordinate**, **Y Coordinate**, and **Location** fields, and functions as a spatial reference lookup source.

No third-party enrichment data was introduced. Instead, the original source data was restructured into prepared sources that better support ETL and dimensional modelling. The hierarchical structure used in the warehouse design was derived directly from the available source attributes.

For example:

- The **Date** field supports time-based hierarchies such as **year, quarter, month, and day**
- Offence-related attributes support **offence categorisation**

As a result, the final source design remains closely aligned with the original operational data while supporting effective analytical processing.

2.1 Data Descriptions

Source Types

Source File	Source Type	Description
crimes_original_reduced.csv	CSV	Main incident-level source file containing one row per crime incident
offense_source.txt	Text (tab-delimited)	Offense lookup source containing distinct IUCR definitions
coordinate_source.xlsx	Excel (.xlsx)	Coordinate lookup source containing spatial reference values

Prepared Crime Source Files

1. Crime Incident Data (crimes_original_reduced.csv)

Column Name	Datatype	Description
ID	bigint	Unique incident ID
Case Number	varchar(30)	Case number linked to the incident
Date	datetime	Date and time when the crime incident occurred
Block	nvarchar(100)	Street block where the incident was recorded
Location Description	nvarchar(100)	Text description of the incident location
Arrest	bit	Indicates whether an arrest was made (True/False)

Domestic	bit	Indicates whether the incident was domestic-related (True/False)
Beat	varchar(10)	Police beat code
District	varchar(10)	Police district code
Ward	varchar(10)	Ward code related to the incident location
Community Area	varchar(10)	Community area code related to the incident location
Year	int	Year extracted from the incident date
Updated On	datetime	Last update timestamp of the source record
Latitude	decimal	Latitude of the incident location
Longitude	decimal	Longitude of the incident location
IUCR	varchar(10)	Illinois Uniform Crime Reporting offense code

2. Offense Definition Data (offense_source.txt)

Column Name	Datatype	Description
IUCR	varchar(10)	Unique offense code used to classify crimes
Primary Type	nvarchar(100)	Main crime category

Description	nvarchar(255)	Detailed description of the offense
FBI Code	varchar(10)	FBI classification code for the offense

3. Coordinate Lookup Data (coordinate_source.xlsx)

Column Name	Datatype	Description
Latitude	decimal	Latitude value of the coordinate point
Longitude	decimal	Longitude value of the coordinate point
X Coordinate	int	X-coordinate reference value
Y Coordinate	int	Y-coordinate reference value
Location	nvarchar(100)	Combined text representation of the geographic location

Task 3: High Level Design Architecture

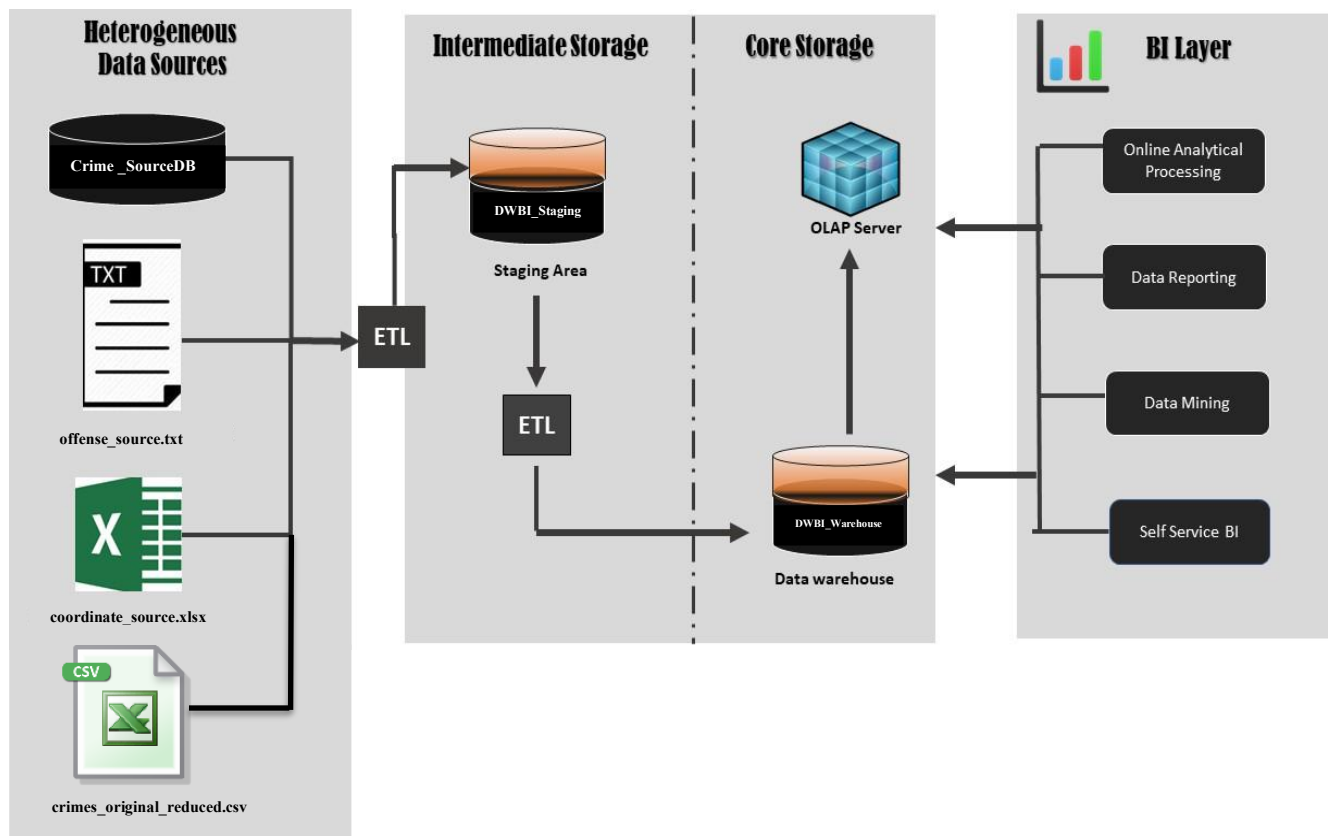


Figure 2: Layered Architecture of the Crime Data Warehouse System

The figure above illustrates the high-level architecture of the proposed crime data warehouse and business intelligence solution. Data is collected from multiple heterogeneous sources, including the Crime_SourceDB source database, offense_source.txt, coordinate_source.xlsx, and crimes_original_reduced.csv. Since real-time data capture is not required, the raw source data is first extracted into the DWBI_Staging database through SSIS.

At the staging layer, ETL operations such as data cleansing, data conversion, lookup processing, and derived-column transformations are carried out before loading the data into the warehouse. The processed data is then stored in DWBI_Warehouse, which contains the dimensional model used for analysis.

Finally, the BI layer supports reporting, OLAP-based analysis, data mining, and self-service BI.

Data Storage Snapshots from SSMS

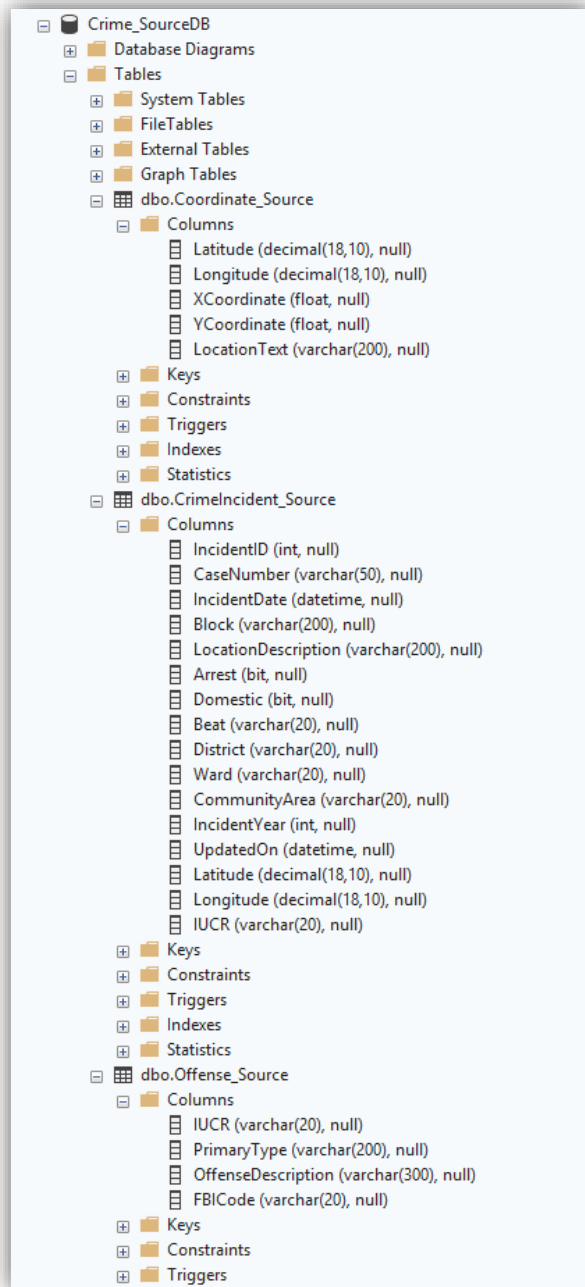


Figure 5: Implementation of the Source in SSMS

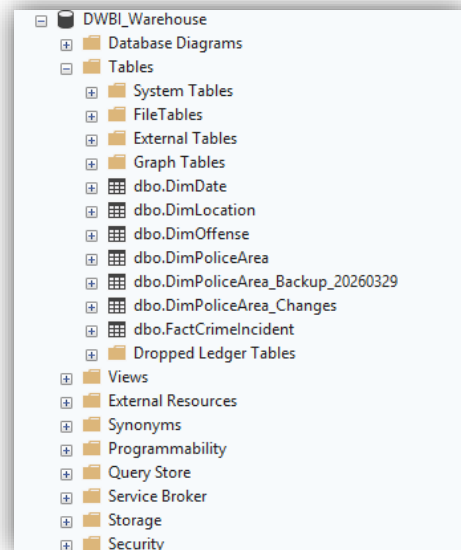


Figure 3: Implementation of the Staging

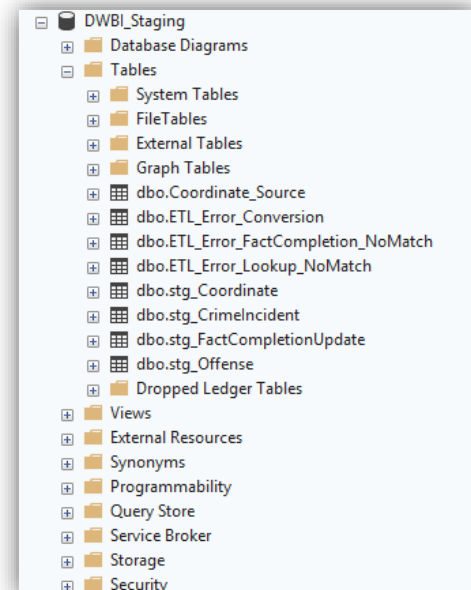


Figure 4: Implementation of the Data Warehouse Databases

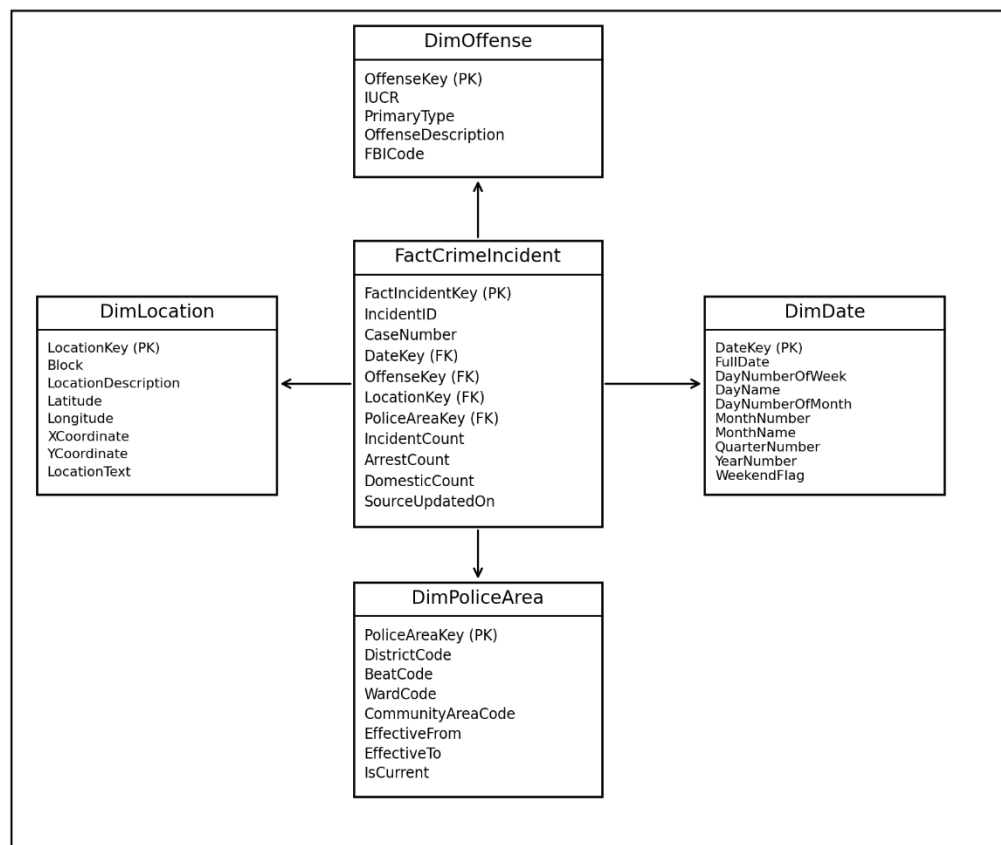
Task 4: Data Warehouse design and Development

4.0 Introduction

A dimensional data warehouse was designed and implemented in SQL Server to support analytical reporting on crime incidents. A star schema was selected because it provides a simple and efficient structure for querying measures across multiple descriptive dimensions. The warehouse consists of **one** fact table, FactCrimeIncident, and **four** dimensions: DimDate, DimOffense, DimLocation, and DimPoliceArea.

4.1 Conceptual Data model

A conceptual model was first developed to identify the major entities and relationships in the dataset. The key analytical entities identified were Crime Incident, Offense, Date, Location, and Police Area. This analysis helped define the final dimensional model and determine which entities should become dimensions and which measures should be stored in the fact table.



4.2 Dimensional Model Design

A star schema was designed and implemented in the DWBI_Warehouse database. The schema includes one fact table and four-dimension tables. The fact table captures incident-level measures, while the dimensions provide descriptive context for analysis.

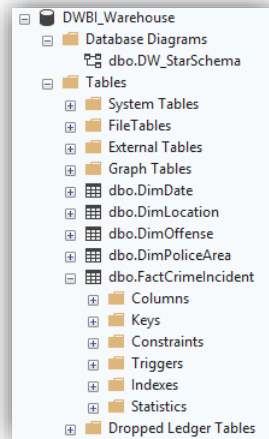


Figure 7: Implemented Fact and Dimension Tables in the Data Warehouse

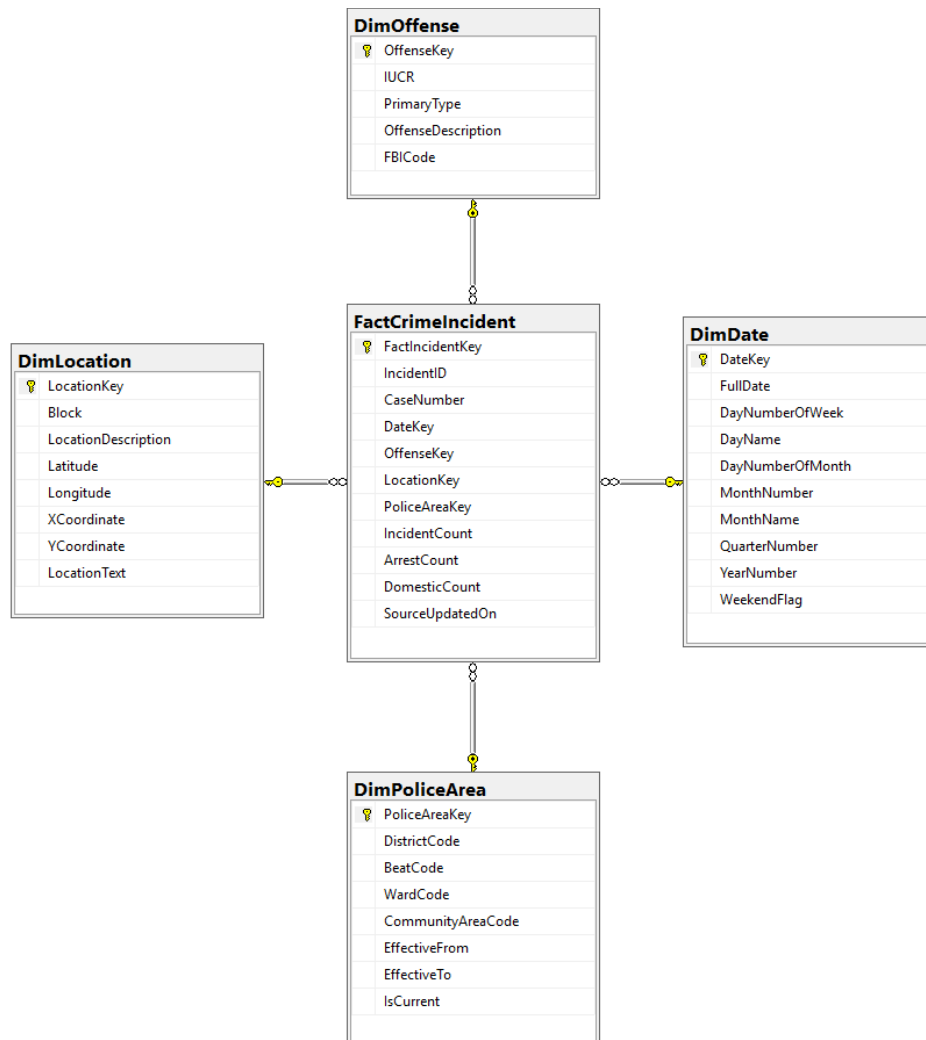


Figure 8: Star Schema Implementation of the Crime Data Warehouse

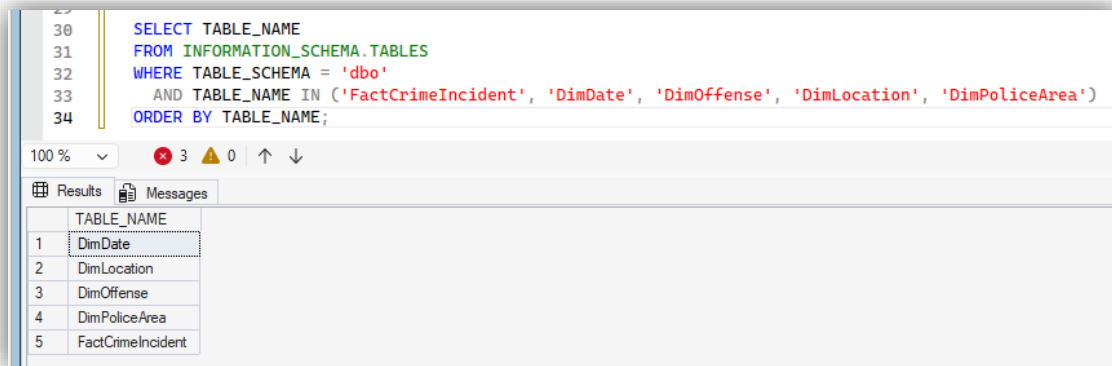


Figure 9: SQL Query Result Showing the Created Fact and Dimension Tables

The central fact table, **FactCrimeIncident**, is directly associated with the following dimensions:

- **DimDate**
- **DimOffense**
- **DimLocation**
- **DimPoliceArea**

The star schema was selected because it provides a clear and efficient structure for analytical processing. Its simplified design supports effective reporting and enables incident records to be examined by time, offense type, location, and police area.

The grain of the fact table is defined as follows:

Each row in **FactCrimeIncident** represents one unique crime incident identified by **IncidentID**.

```

29
30 SELECT TABLE_NAME
31 FROM INFORMATION_SCHEMA.TABLES
32 WHERE TABLE_SCHEMA = 'dbo'
33 AND TABLE_NAME IN ('FactCrimeIncident', 'DimDate', 'DimOffense', 'DimLocation', 'DimPoliceArea')
34 ORDER BY TABLE_NAME;
35
36
37 SELECT TOP 20
38     FactIncidentKey,
39     IncidentID,
40     CaseNumber,
41     DateKey,
42     OffenseKey,
43     LocationKey,
44     PoliceAreaKey,
45     IncidentCount,
46     ArrestCount,
47     DomesticCount
48 FROM dbo.FactCrimeIncident
49 ORDER BY FactIncidentKey;

```

100 % 3 0

Results Messages

	TABLE_NAME
1	DimDate
2	DimLocation
3	DimOffense
4	DimPoliceArea
5	FactCrimeIncident

	FactIncidentKey	IncidentID	CaseNumber	DateKey	OffenseKey	LocationKey	PoliceAreaKey	IncidentCount	ArrestCount	DomesticCount
1	3795806	27602	JG322799	20230630	17	294482	6135	1	0	0
2	3795807	28028	JH189071	20240315	17	268082	6278	1	0	0
3	3795808	28089	JH239323	20240426	17	21516	6270	1	0	0
4	3795809	27674	JG368774	20230408	17	224263	6000	1	0	0
5	3795810	27675	JG369252	20230408	17	395281	6129	1	0	0
6	3795811	28626	JJ176329	20251003	17	255784	6014	1	0	0
7	3795812	27449	JG235011	20230424	17	386285	6218	1	1	0
8	3795813	27505	JG265892	20230518	17	284838	6280	1	0	0
9	3795814	27522	JG277162	20230527	17	297960	6032	1	0	0
10	3795815	27732	JG393002	20230409	17	400858	6183	1	0	0
11	3795816	28001	JH170910	20240229	17	204380	6000	1	0	0
12	3795817	28701	JJ258837	20250517	17	391893	6173	1	0	0
13	3795818	27354	JG161306	20230221	17	311430	6141	1	1	0

Figure 10: Query Result Displaying Sample Data from the Fact Table

```

52 SELECT
53     COUNT(*) AS TotalFactRows,
54     COUNT(DISTINCT IncidentID) AS DistinctIncidentIDs
55 FROM dbo.FactCrimeIncident;

```

100 % 3 0

Results Messages

	TotalFactRows	DistinctIncidentIDs
1	759161	759161

Figure 11: Aggregate Verification of Records in the FactCrimeIncident Table

The total number of fact rows matches the number of distinct IncidentID values, confirming that each fact row represents one unique incident.

4.3 Fact Table Description

FactCrimeIncident is the central fact table of the schema. It stores one row per crime incident and contains foreign keys to all relevant dimensions. It also stores analytical measures such as incident count, arrest count, and domestic count.

Foreign keys

- DateKey
- OffenseKey
- LocationKey
- PoliceAreaKey

Degenerate/natural identifiers

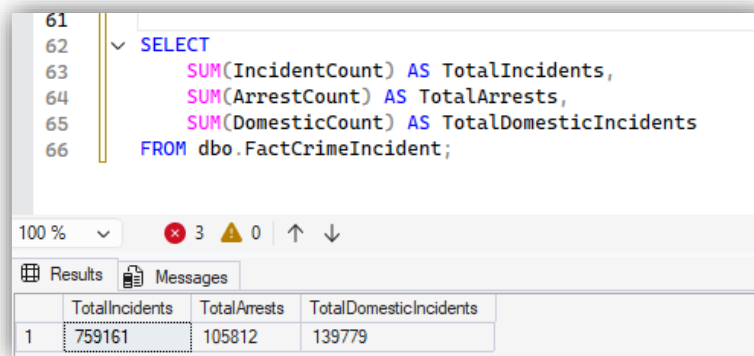
- IncidentID
- CaseNumber

Measures

- IncidentCount
- ArrestCount
- DomesticCount

Audit attribute

- SourceUpdatedOn



The screenshot shows a SQL query window with the following text:

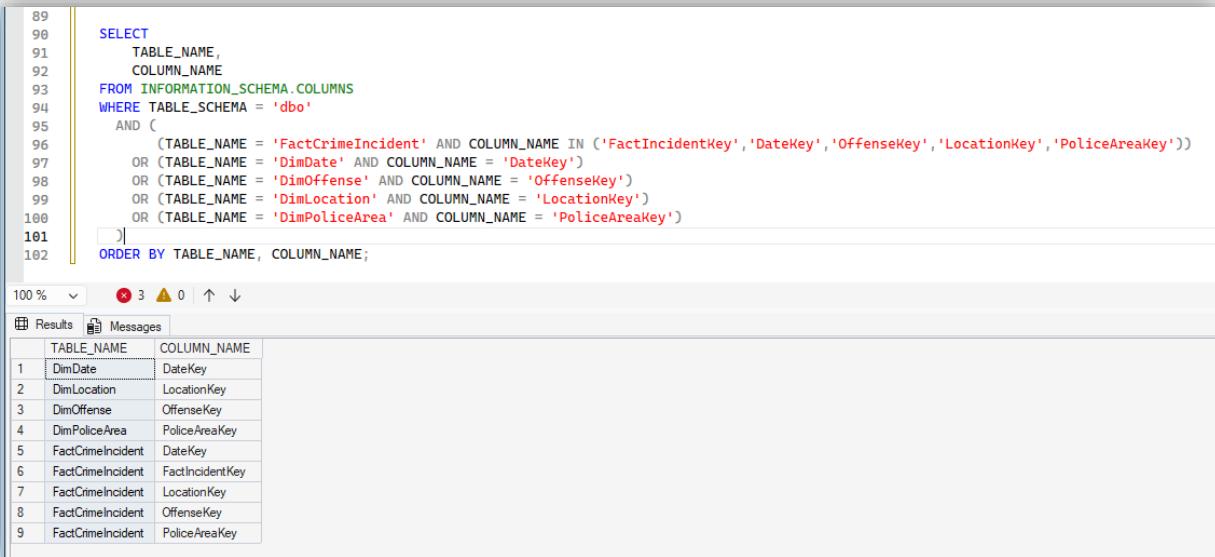
```
61  
62  SELECT  
63      SUM(IncidentCount) AS TotalIncidents,  
64      SUM(ArrestCount) AS TotalArrests,  
65      SUM(DomesticCount) AS TotalDomesticIncidents  
66  FROM dbo.FactCrimeIncident;
```

Below the query window, the 'Results' tab is active, displaying a single row of data:

	TotalIncidents	TotalArrests	TotalDomesticIncidents
1	759161	105812	139779

Figure 12: Validation of Summarized Measures in the FactCrimeIncident Table

4.4 Dimension Table Description



```
89 SELECT
90     TABLE_NAME,
91     COLUMN_NAME
92 FROM INFORMATION_SCHEMA.COLUMNS
93 WHERE TABLE_SCHEMA = 'dbo'
94 AND (
95     (TABLE_NAME = 'FactCrimeIncident' AND COLUMN_NAME IN ('FactIncidentKey', 'DateKey', 'OffenseKey', 'LocationKey', 'PoliceAreaKey'))
96     OR (TABLE_NAME = 'DimDate' AND COLUMN_NAME = 'DateKey')
97     OR (TABLE_NAME = 'DimOffense' AND COLUMN_NAME = 'OffenseKey')
98     OR (TABLE_NAME = 'DimLocation' AND COLUMN_NAME = 'LocationKey')
99     OR (TABLE_NAME = 'DimPoliceArea' AND COLUMN_NAME = 'PoliceAreaKey')
100 )
101
102 ORDER BY TABLE_NAME, COLUMN_NAME;
```

	TABLE_NAME	COLUMN_NAME
1	DimDate	DateKey
2	DimLocation	LocationKey
3	DimOffense	OffenseKey
4	DimPoliceArea	PoliceAreaKey
5	FactCrimeIncident	DateKey
6	FactCrimeIncident	FactIncidentKey
7	FactCrimeIncident	LocationKey
8	FactCrimeIncident	OffenseKey
9	FactCrimeIncident	PoliceAreaKey

Figure 13: Query Result Confirming Key Columns in Fact and Dimension Tables

4.4.1 DimDate

- Purpose: time-based analysis
- Important attributes: FullDate, DayName, MonthName, QuarterNumber, YearNumber, WeekendFlag

4.4.2 DimOffense

- Purpose: offense analysis
- Key attributes: IUCR, PrimaryType, OffenseDescription, FBI Code

4.4.3 DimLocation

- Purpose: spatial/location analysis
- Attributes: Block, LocationDescription, Latitude, Longitude, XCoordinate, YCoordinate, LocationText

4.4.4 DimPoliceArea

DimPoliceArea was implemented as a **Slowly Changing Dimension Type 2 (SCD Type 2)** to preserve historical changes in police area attributes over time.

```

122 SELECT TOP 20
123     PoliceAreaKey,
124     DistrictCode,
125     BeatCode,
126     WardCode,
127     CommunityAreaCode,
128     EffectiveFrom,
129     EffectiveTo,
130     IsCurrent
131 FROM dbo.DimPoliceArea
132 ORDER BY PoliceAreaKey;

```

	PoliceAreaKey	DistrictCode	BeatCode	WardCode	CommunityAreaCode	EffectiveFrom	EffectiveTo	IsCurrent
1	-1	-1	-1	-1	-1	2026-03-29 01:41:41	9999-12-31 00:00:00	1
2	0	UNK	UNK	NULL	NULL	1900-01-01 00:00:00	9999-12-31 00:00:00	1
3	5994	1	111	34	32	2026-03-29 16:05:08	9999-12-31 00:00:00	1
4	5995	1	124	28	28	2026-03-29 16:05:08	9999-12-31 00:00:00	1
5	5996	1	132	3	33	2026-03-29 16:05:08	9999-12-31 00:00:00	1
6	5997	10	1014	24	29	2026-03-29 16:05:08	9999-12-31 00:00:00	1
7	5998	10	1023	25	30	2026-03-29 16:05:08	9999-12-31 00:00:00	1
8	5999	11	1112	27	23	2026-03-29 16:05:08	9999-12-31 00:00:00	1
9	6000	11	1122	37	26	2026-03-29 16:05:08	9999-12-31 00:00:00	1
10	6001	11	1131	24	25	2026-03-29 16:05:08	9999-12-31 00:00:00	1
11	6002	11	1133	24	27	2026-03-29 16:05:08	9999-12-31 00:00:00	1
12	6003	12	1221	27	23	2026-03-29 16:05:08	9999-12-31 00:00:00	1
13	6004	12	1223	27	28	2026-03-29 16:05:08	9999-12-31 00:00:00	1
14	6005	12	1233	28	28	2026-03-29 16:05:08	9999-12-31 00:00:00	1
15	6006	12	1234	25	31	2026-03-29 16:05:08	9999-12-31 00:00:00	1
16	6007	14	1412	35	22	2026-03-29 16:05:08	9999-12-31 00:00:00	1
17	6008	14	1431	1	22	2026-03-29 16:05:08	9999-12-31 00:00:00	1
18	6009	14	1432	32	22	2026-03-29 16:05:08	9999-12-31 00:00:00	1
19	6010	14	1434	32	22	2026-03-29 16:05:08	9999-12-31 00:00:00	1
20	6011	14	1713	33	14	2026-03-29 16:05:08	9999-12-31 00:00:00	1

Figure 14: SQL Query Output Showing the Police Area Dimension Data

- **Business key:** DistrictCode + BeatCode
- **Changing attributes:** WardCode, CommunityAreaCode
- **SCD tracking columns:** EffectiveFrom, EffectiveTo, IsCurrent

When WardCode or CommunityAreaCode changes for the same DistrictCode + BeatCode, the previous current record is expired by updating EffectiveTo and IsCurrent, and a new current record is inserted.

SCD Example

A sample police-area record was checked to confirm that the latest row is marked as current and that historical changes are preserved.

```

134 SELECT *
135 FROM dbo.DimPoliceArea
136 WHERE DistrictCode = '12'
137 AND BeatCode = '1211'
138 ORDER BY EffectiveFrom DESC;

```

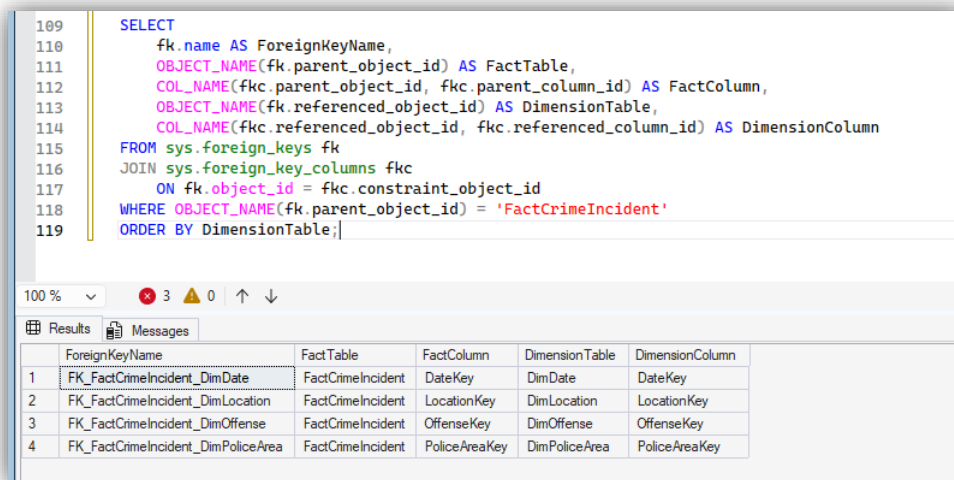
	PoliceAreaKey	DistrictCode	BeatCode	WardCode	CommunityAreaCode	EffectiveFrom	Effective To	IsCurrent
1	6192	12	1211	26	24	2026-03-29 16:05:08	9999-12-31 00:00:00	1

Figure 15: SQL-Based Verification of a Specific District–Beat Record in the DimPoliceArea Dimension Table

The query result for DistrictCode = 12 and BeatCode = 1211 shows the current active version of the police-area dimension record. The row has IsCurrent = 1 and an open-ended EffectiveTo value of 9999-12-31, confirming that the SCD Type 2 implementation correctly maintains the latest version of the record while preserving historical tracking.

Fact-to-dimension relationships

The fact table is linked to each dimension using foreign keys, allowing incident analysis by date, offense, location, and police area



```

109 SELECT
110     fk.name AS ForeignKeyName,
111     OBJECT_NAME(fk.parent_object_id) AS FactTable,
112     COL_NAME(fkc.parent_object_id, fkc.parent_column_id) AS FactColumn,
113     OBJECT_NAME(fk.referenced_object_id) AS DimensionTable,
114     COL_NAME(fkc.referenced_object_id, fkc.referenced_column_id) AS DimensionColumn
115 FROM sys.foreign_keys fk
116 JOIN sys.foreign_key_columns fkc
117     ON fk.object_id = fkc.constraint_object_id
118 WHERE OBJECT_NAME(fk.parent_object_id) = 'FactCrimeIncident'
119 ORDER BY DimensionTable;

```

	ForeignKeyName	FactTable	FactColumn	DimensionTable	DimensionColumn
1	FK_FactCrimeIncident_DimDate	FactCrimeIncident	DateKey	DimDate	DateKey
2	FK_FactCrimeIncident_DimLocation	FactCrimeIncident	LocationKey	DimLocation	LocationKey
3	FK_FactCrimeIncident_DimOffense	FactCrimeIncident	OffenseKey	DimOffense	OffenseKey
4	FK_FactCrimeIncident_DimPoliceArea	FactCrimeIncident	PoliceAreaKey	DimPoliceArea	PoliceAreaKey

Figure 16: Verification of Foreign Key Constraints in the Implemented Star Schema

4.5 Keys and Relationships

Surrogate keys

- DateKey
- OffenseKey
- LocationKey
- PoliceAreaKey
- FactIncidentKey

Foreign key relationships

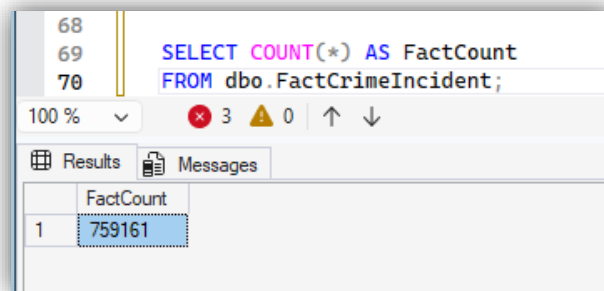
- FactCrimeIncident.DateKey → DimDate.DateKey
- FactCrimeIncident.OffenseKey → DimOffense.OffenseKey
- FactCrimeIncident.LocationKey → DimLocation.LocationKey
- FactCrimeIncident.PoliceAreaKey → DimPoliceArea.PoliceAreaKey

Surrogate keys were used in all dimensions to provide stable warehouse keys independent of changes in source-system values and to support efficient joins between the fact and dimension tables.

4.6 Design Assumptions

- Each row in the fact table represents a single crime incident
- IncidentCount is always stored as 1
- DistrictCode + BeatCode was treated as the business key for police area
- WardCode and CommunityAreaCode may change over time and were therefore tracked using SCD Type 2
- Unknown/default members were used where necessary for unmatched dimension values

4.7 Validation of the Dimensional Model



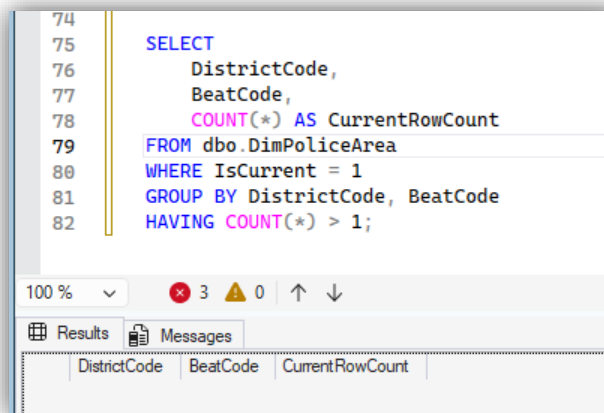
The screenshot shows a SQL query window with the following text:

```
68  
69 SELECT COUNT(*) AS FactCount  
70 FROM dbo.FactCrimeIncident;
```

Below the query, the status bar indicates 100% zoom, 3 errors, and 0 warnings. The Results tab is active, showing a single row with the value 759161 for the FactCount column.

FactCount
759161

Figure 17: SQL-Based Verification of the Total Number of Records in the FactCrimeIncident Table



The screenshot shows a SQL query window with the following text:

```
74  
75 SELECT  
76     DistrictCode,  
77     BeatCode,  
78     COUNT(*) AS CurrentRowCount  
79 FROM dbo.DimPoliceArea  
80 WHERE IsCurrent = 1  
81 GROUP BY DistrictCode, BeatCode  
82 HAVING COUNT(*) > 1;
```

Below the query, the status bar indicates 100% zoom, 3 errors, and 0 warnings. The Results tab is active, showing a table with columns DistrictCode, BeatCode, and CurrentRowCount. No rows are displayed, confirming no duplicates.

DistrictCode	BeatCode	CurrentRowCount
--------------	----------	-----------------

Figure 18: SQL Query Result Confirming No Duplicate Current Police Area Records

The first query confirms that FactCrimeIncident was successfully populated with **759,161** rows. The second query validates the SCD Type 2 implementation in DimPoliceArea by checking for duplicate current rows for the same business key (DistrictCode + BeatCode). Since no rows were returned, the dimension correctly maintains only one current row per business key.

Task 5: ETL Development

5.0 Overview of the ETL process

The ETL process was developed in SQL Server Integration Services (SSIS) to extract data from multiple source types, transform and cleanse the extracted data, load it into staging tables, populate the dimension tables, and finally load the fact table. The ETL was designed in a controlled sequence so that staging tables were loaded first, followed by dimensions, and finally the fact table. This loading order is important because the fact table depends on surrogate keys generated in the dimension tables.

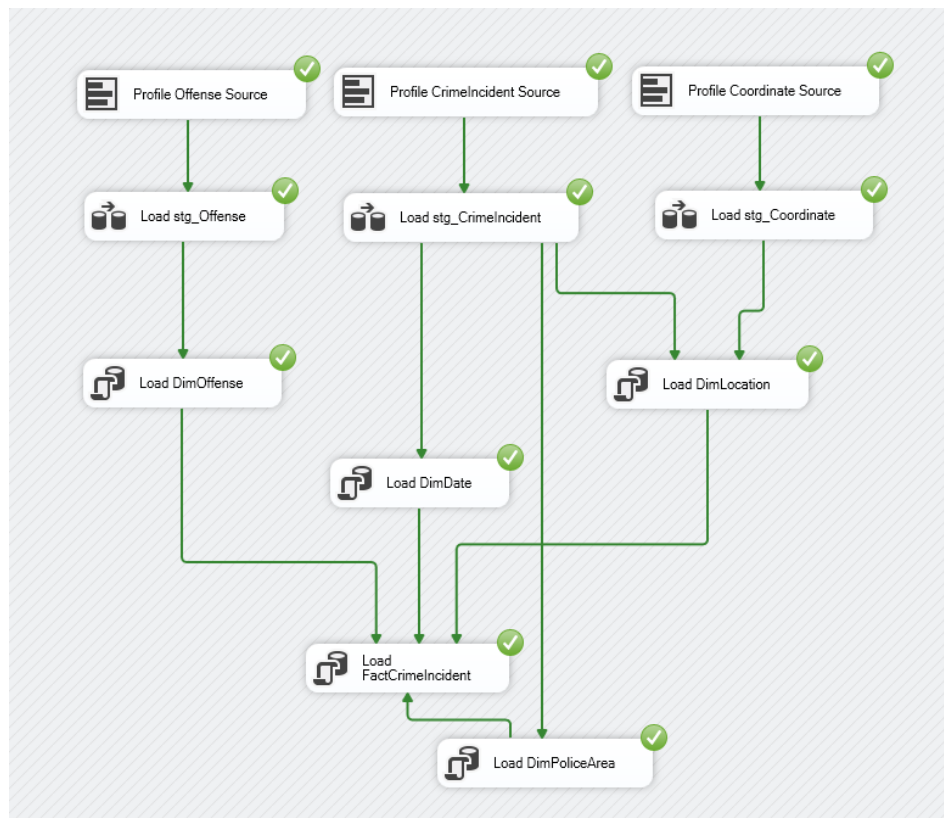


Figure 19: SSIS Data Flow for Populating Fact and Dimension Tables

The control flow demonstrates the execution sequence of the SSIS package. The package first loads the staging tables (Load_stg_Offense, Load_stg_CrimeIncident, Load_stg_Coordinate), then loads the dimensions (Load DimOffense, Load DimDate, Load DimLocation, Load DimPoliceArea), and finally loads the fact table (Load FactCrimeIncident). This execution order ensures that the required surrogate keys are available before fact loading begins.

5.1 Data Profiling

Before the main ETL execution, source data profiling was carried out to better understand the structure, completeness, and quality of the source data. This step was used to identify issues such as null values, duplicate business keys, and value distributions in important source attributes before loading the data into the staging layer. In this implementation, profiling checks were applied to key columns such as IUCR, District, Beat, Latitude, and Longitude. The results of this profiling step were then used to support later ETL activities such as data conversion, lookup validation, and error-handling logic in SSIS.

From the images below SSIS **Control Flow** shows the data profiling tasks before the staging loads, the **Data Profiling Task Editor** shows configured profile requests, and the **profile output / viewer results** or sample validation results demonstrating the profiling findings.

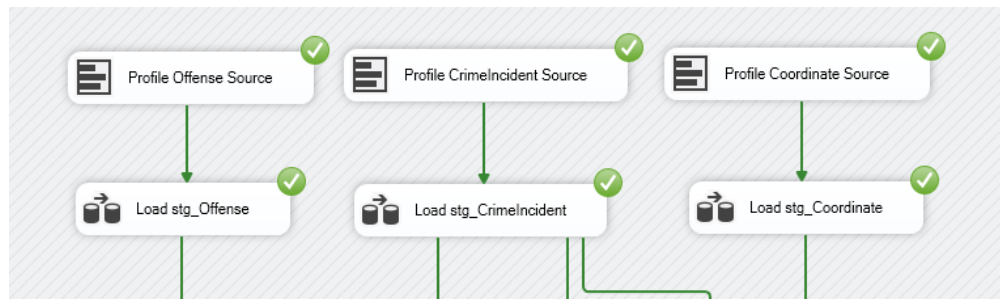
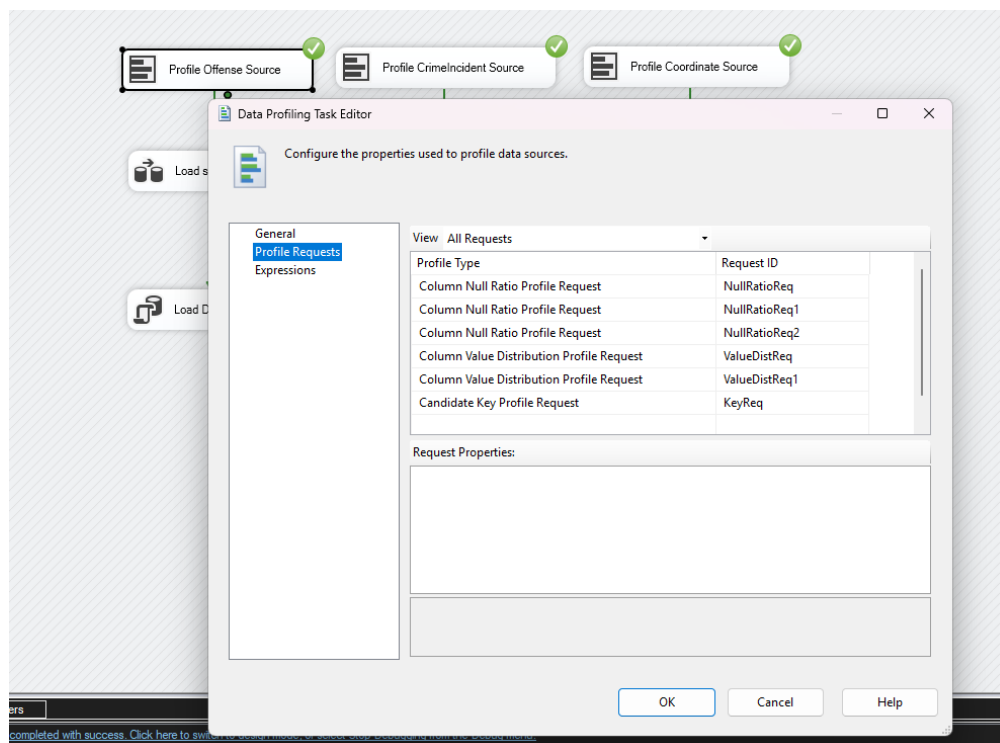


Figure 20: Initial ETL Steps for Profiling Source Data and Loading Staging Tables



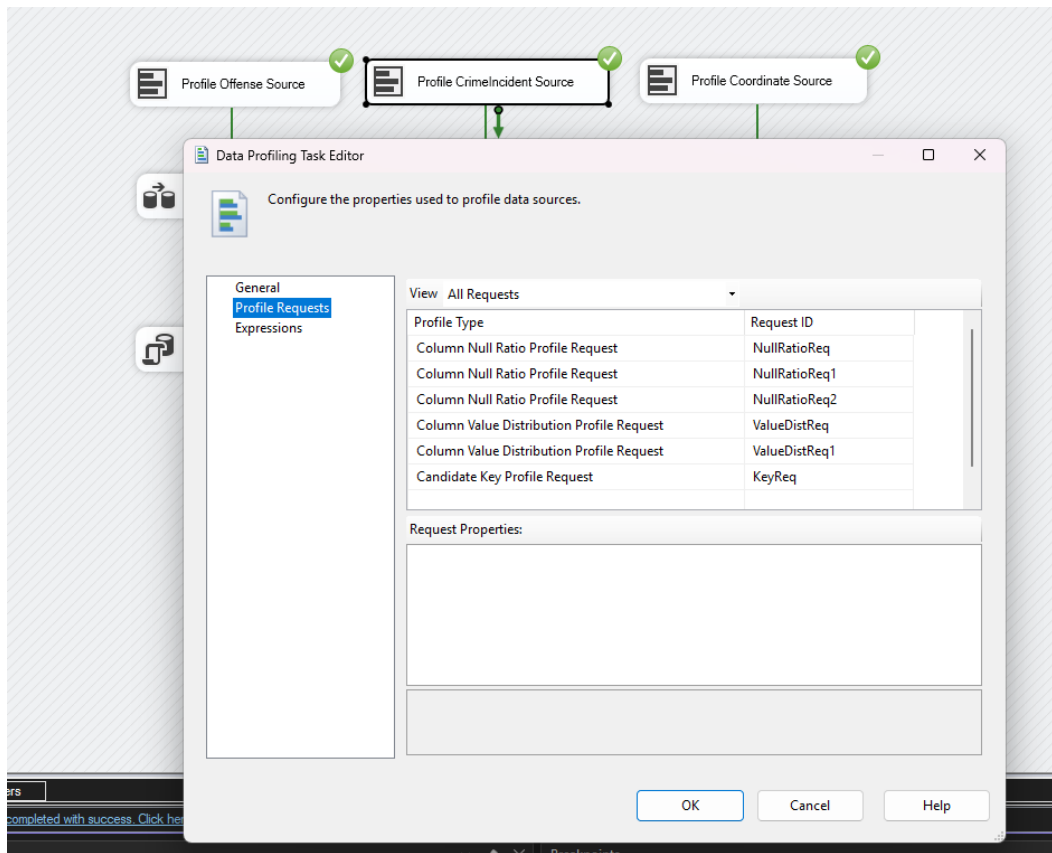
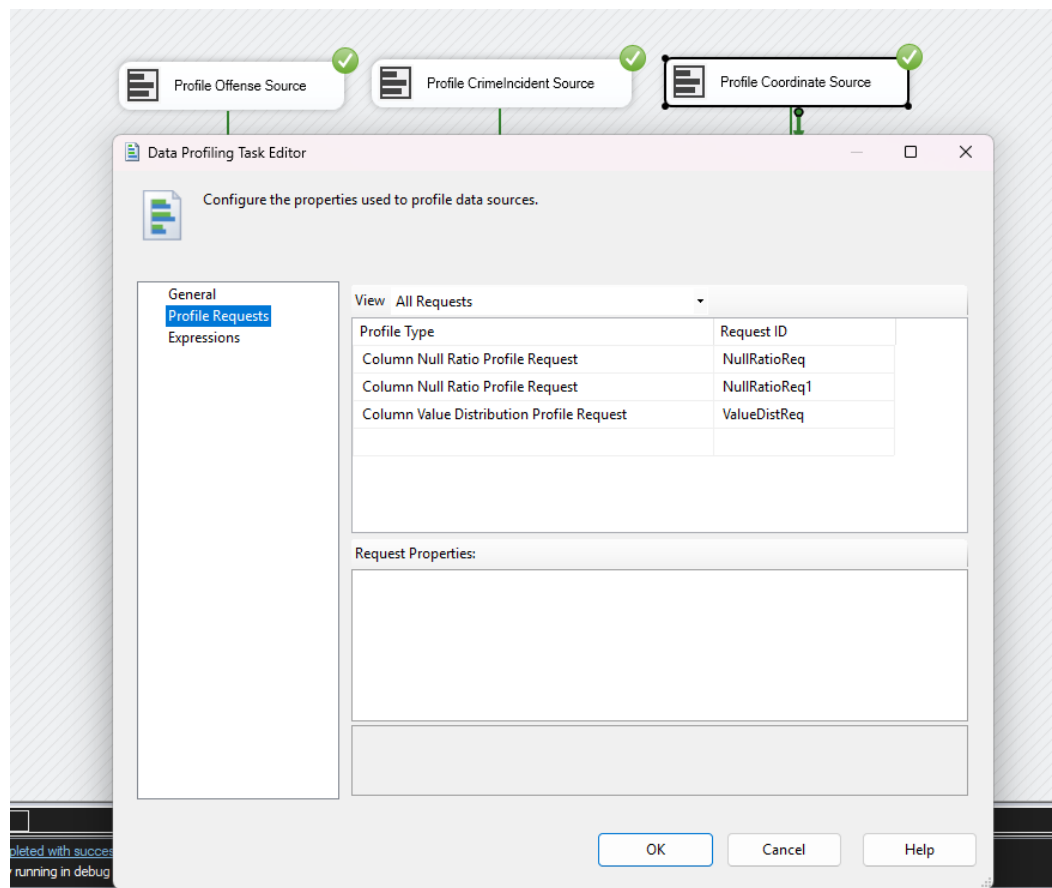


Figure 23: Configuration of Data Profiling Requests in SSIS for Assessing the CrimeIncident Source Data



5.2 Use of Multiple Source Types and ETL Extraction

The ETL solution uses 3 source types. In this implementation:

- **Offense data** is loaded from a **flat file source** using the connection manager **CM_OffenseTXT**
- **Crime incident data** is loaded from a **flat file source** using the connection manager **CM_IncidentCSV**
- **Coordinate data** is loaded from an **Excel source** using the connection manager **CM_CoordinateXLSX**
- **SQL Server** is used as the destination for both the **staging database** and the **data warehouse**

This shows that the ETL process is able to integrate data from **heterogeneous sources** into one dimensional warehouse, which strengthens the overall ETL design.

The implemented source extraction method is **full extraction**, since the ETL package reads the complete source files during each run rather than extracting only changed records.

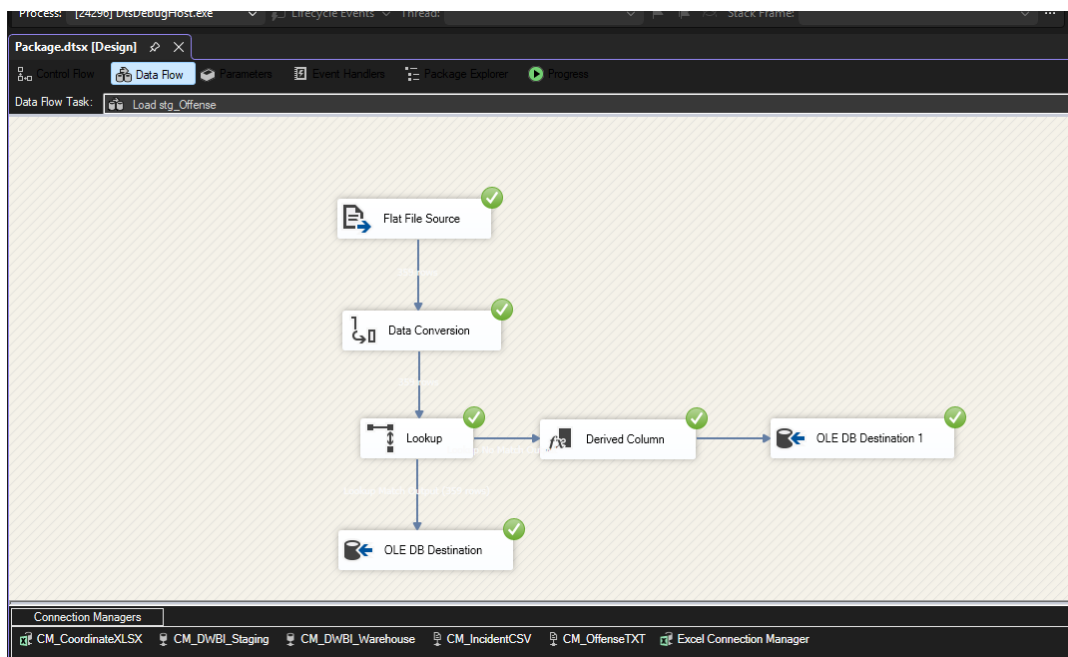


Figure 24: SSIS-Based ETL Data Flow for Transforming and Loading Data into the DimOffense Table

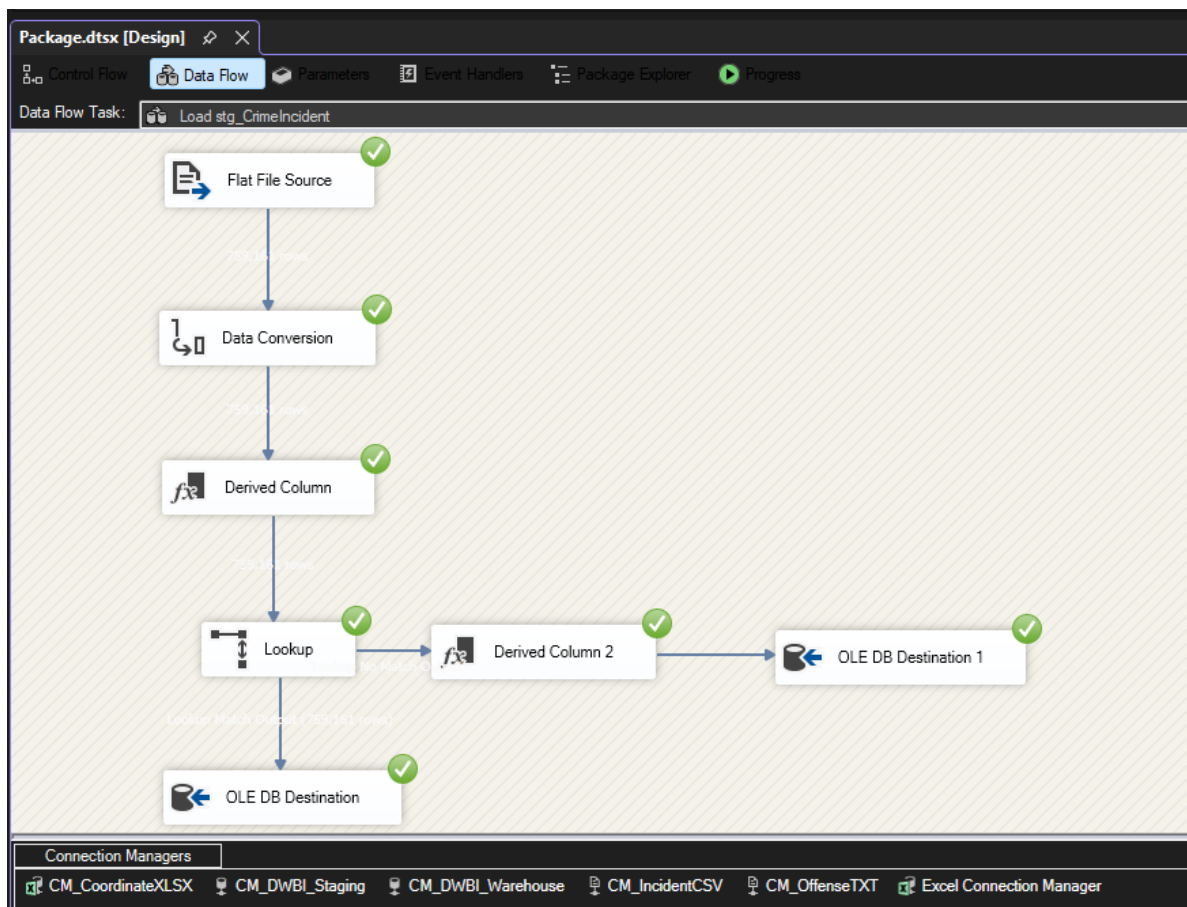


Figure 25: SSIS-Based ETL Data Flow for Transforming and Loading Data into the stg_CrimeIncident Table

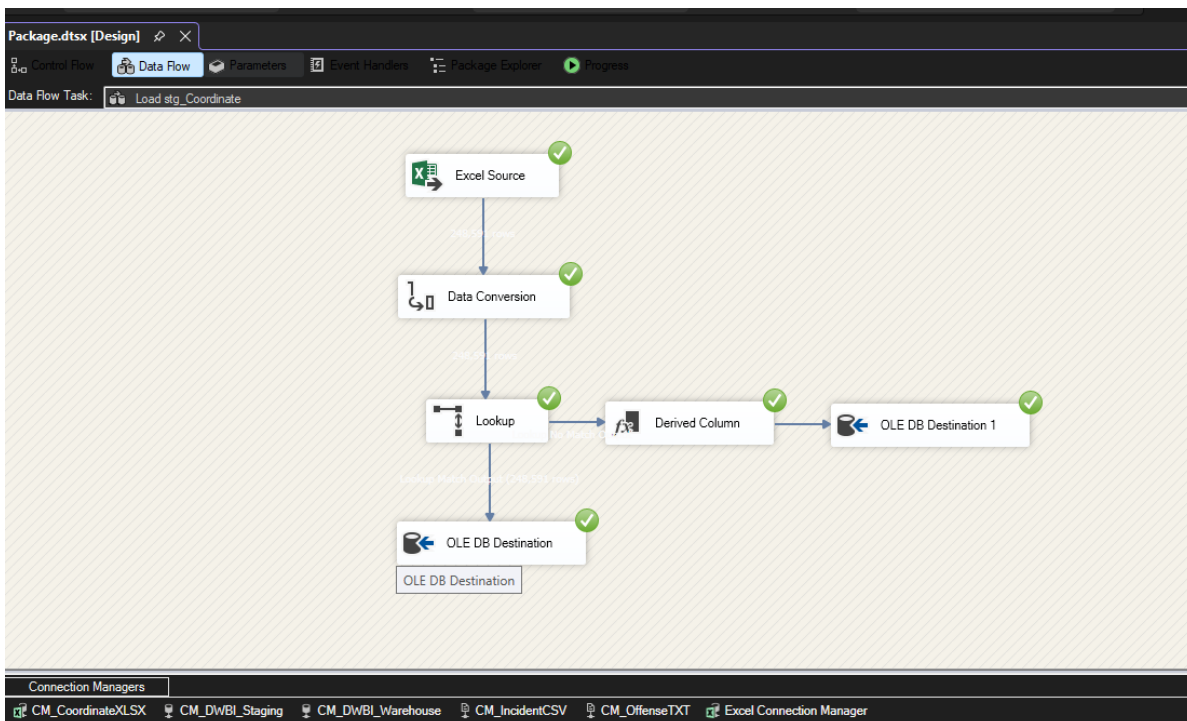


Figure 26: SSIS-Based ETL Data Flow for Transforming and Loading Data into the stg_Coordinate Table

5.3 ETL Transformations

Several SSIS transformations were used throughout the ETL process to improve data quality, resolve keys, apply business logic, and support reliable loading. These transformations were not used merely for package completion; each one served a distinct role in the ETL pipeline.

5.3.1 Data Conversion

The **Data Conversion** transformation was used to convert source columns into data types compatible with SQL Server destination tables. This was necessary because source files often contain values as strings or text-based representations, while the staging and warehouse tables require structured SQL data types such as integers, dates, and numeric attributes. By converting these values before loading, datatype mismatches were avoided and the package became more reliable during execution.

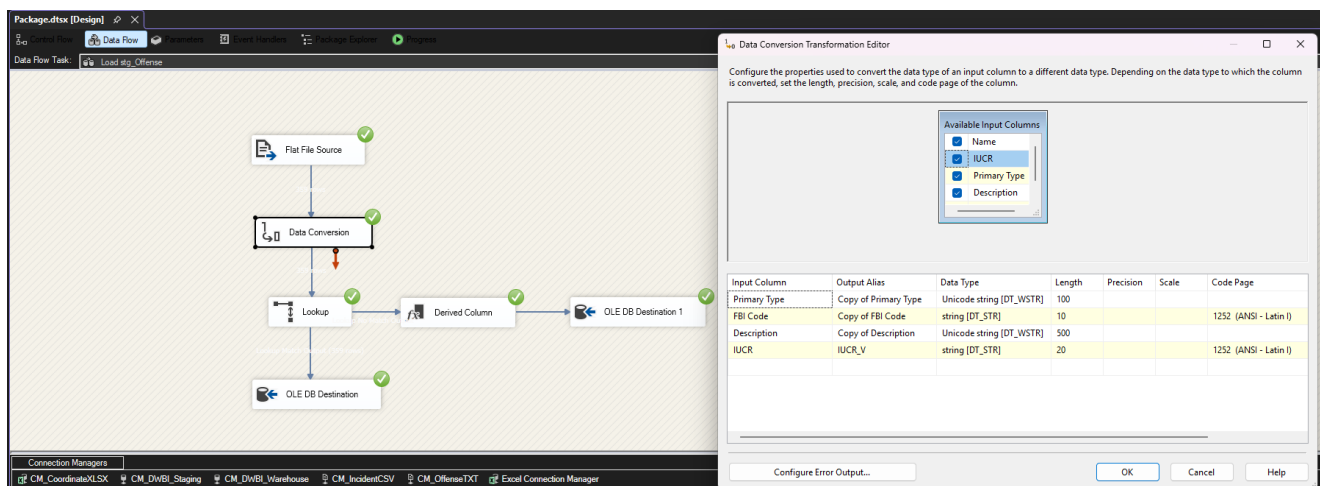


Figure 27: Configuration of the Data Conversion Transformation in the SSIS Offense Loading Process

The above image shows the **Load_stg_Offense** data flow with the **Data Conversion** transformation open. The transformation is used to convert selected input columns, such as **Primary Type**, **FBI Code**, **Description**, and **IUCR**, into the required SSIS data types before loading. This step helps ensure that the source flat file data matches the expected format for subsequent transformations and SQL Server loading.

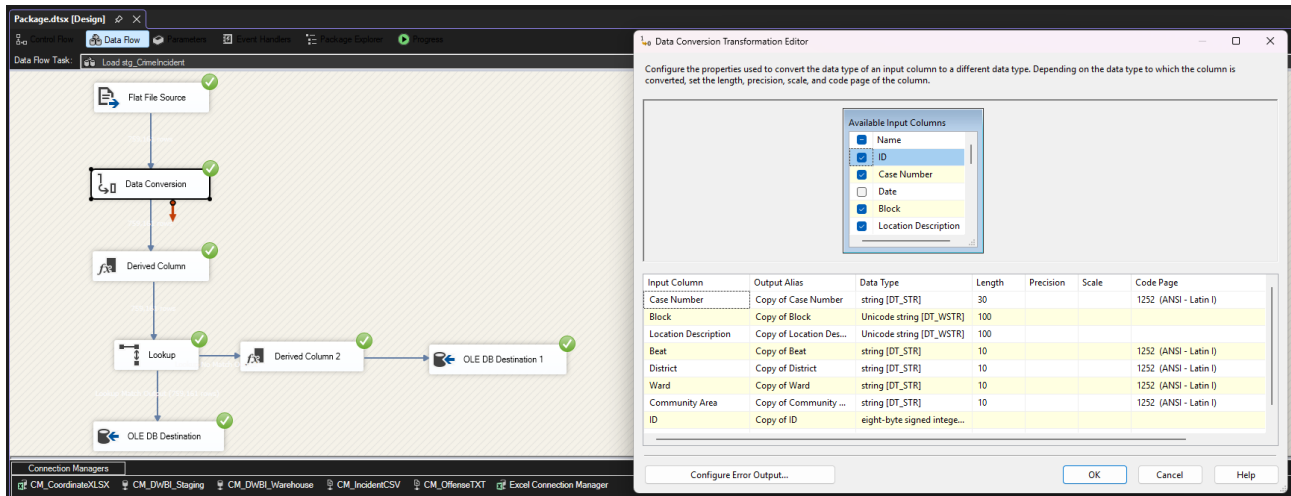


Figure 28: Configuration of the Data Conversion Transformation in the SSIS CrimeIncident Loading Process

The **Data Conversion** transformation was also applied in the **Load_stg_CrimeIncident** data flow to convert source attributes into data types suitable for SQL Server loading. As shown in the above image, columns including **Case Number**, **Block**, **Location Description**, **Beat**, **District**, **Ward**, **Community Area**, and **ID** were converted into the required SSIS data types. This helped to avoid datatype mismatch errors and ensured smoother loading into the staging and warehouse tables.

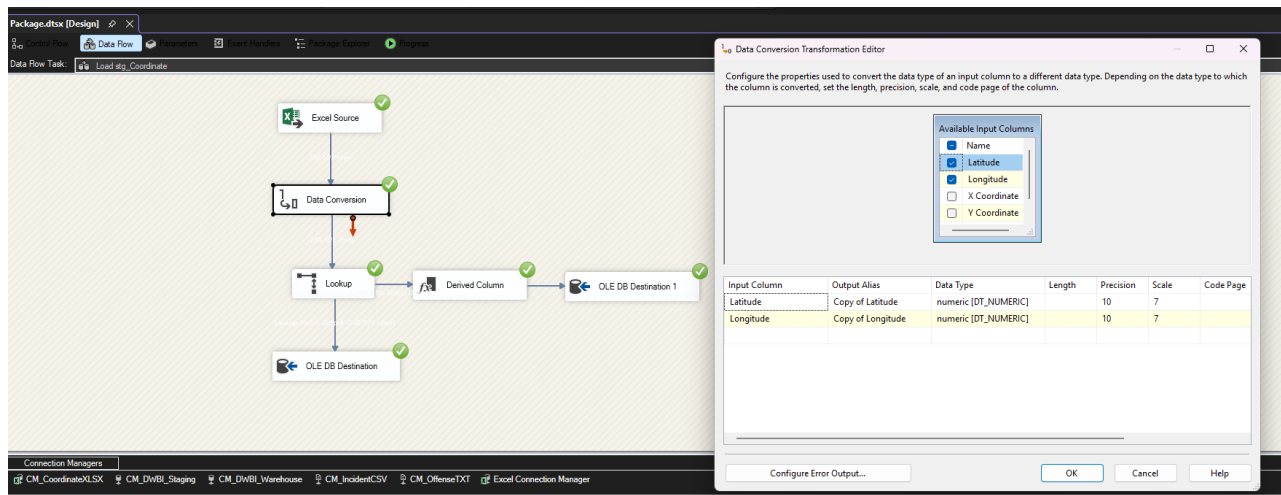


Figure 29: Configuration of the Data Conversion Transformation in the SSIS Coordinate Loading Process

The **Data Conversion** transformation was also used in the **Load_stg_Coordinate** data flow to convert geographical attributes from the Excel source into SQL-compatible numeric formats. As shown in the figure above, **Latitude** and **Longitude** were converted to **numeric [DT_NUMERIC]** values with defined precision and scale. This was necessary to preserve the accuracy of coordinate data and to ensure successful loading into the staging and warehouse tables.

5.3.2 Lookup

The **Lookup** transformation was used to compare incoming rows against existing reference or destination data. In staging flows, the lookup helps identify whether a record already exists and whether it should be inserted or redirected. In warehouse loading, lookups are used to resolve business keys to surrogate keys, which is necessary for correct fact-to-dimension linking. The lookup also supports row routing by separating matched rows from no-match rows.

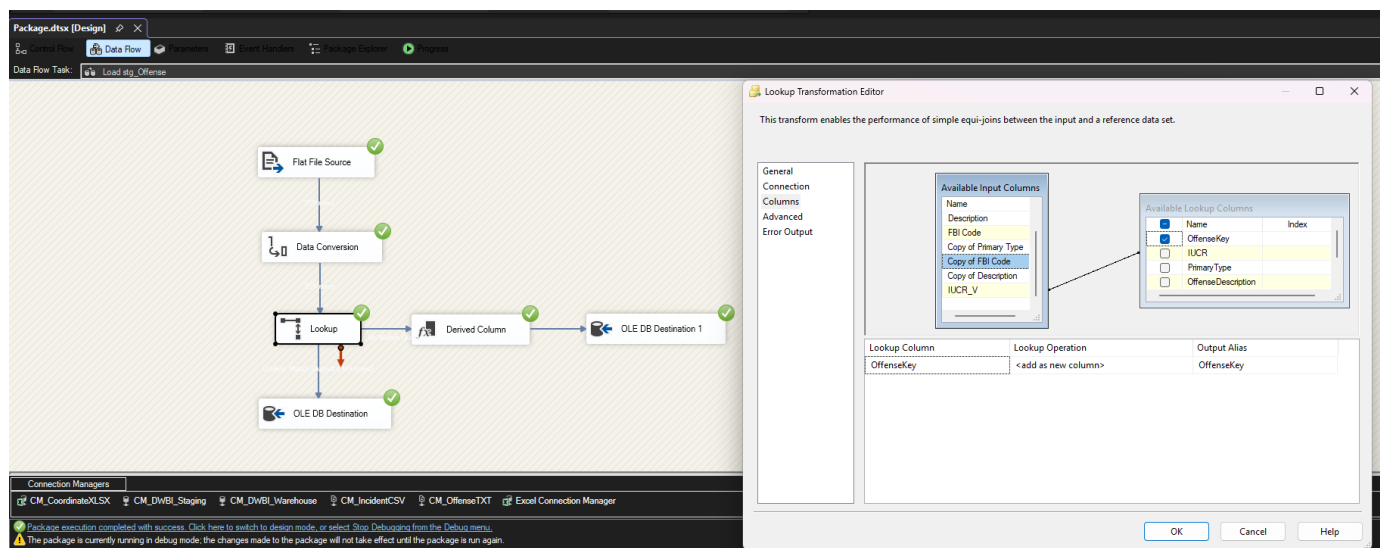


Figure 30: Configuration of the Lookup Transformation in the SSIS Offense Loading Process

The **Lookup** transformation was used in the **Load_stg_Offense** data flow to match source records with existing reference data in the warehouse. As shown in the above figure, the transformation retrieves the **OffenseKey** from the lookup table by comparing input columns with the corresponding offense attributes. This step is important because it enables the ETL process to assign the correct surrogate key to each record before loading, thereby supporting referential integrity between fact and dimension tables.

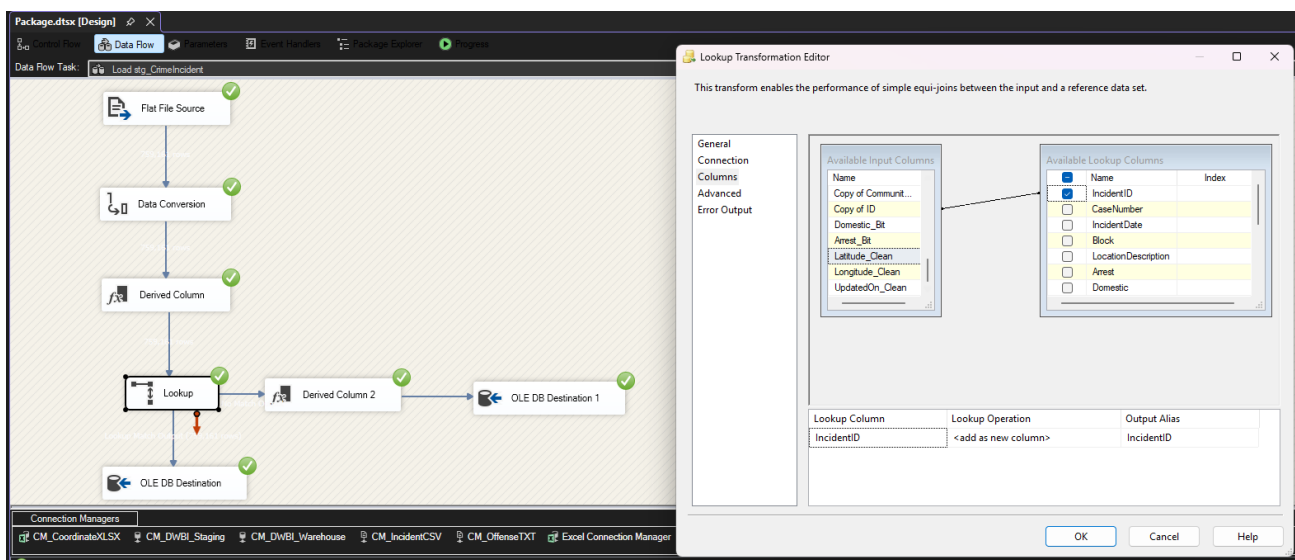


Figure 31: Configuration of the Lookup Transformation in the SSIS CrimeIncident Loading Process

The **Lookup** transformation was also used in the **Load_stg_CrimeIncident** data flow to match incoming crime incident records with reference data stored in SQL Server. As shown in the above image, the transformation returns **IncidentID** as a new column after matching the selected input attributes with the lookup table. This step supports accurate record identification, improves consistency in the ETL process, and helps maintain integrity during loading into the staging and warehouse tables.

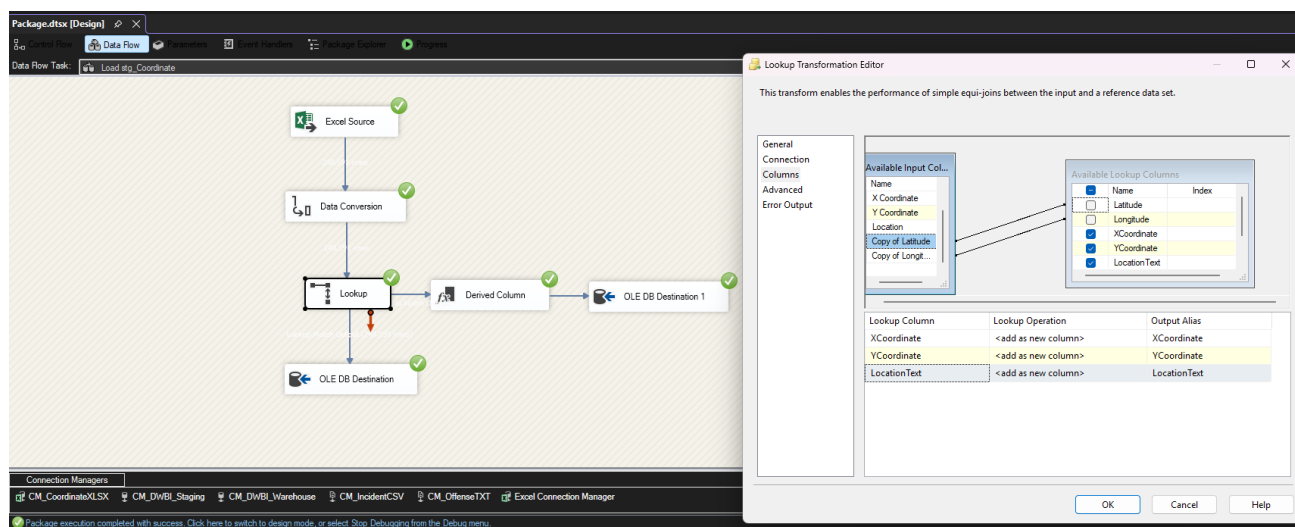


Figure 32: Configuration of the Lookup Transformation in the SSIS Coordinate Loading Process

The **Lookup** transformation was used in the **Load_stg_Coordinate** data flow to compare incoming coordinate records from the Excel source with reference data stored in SQL Server. As shown in the image above, the transformation returns **XCoordinate**, **YCoordinate**, and **LocationText** as new columns after matching

the selected input attributes with the lookup table. This step improves data consistency, supports more accurate location-related loading, and helps maintain integrity in the staging and warehouse process.

5.3.3 Derived Column

The **Derived Column** transformation was used to generate calculated values and ETL metadata. This includes creating business-rule fields, assigning default values, and generating audit-related columns such as package name, task name, lookup name, and error descriptions. This transformation improves traceability and makes rejected rows easier to investigate.

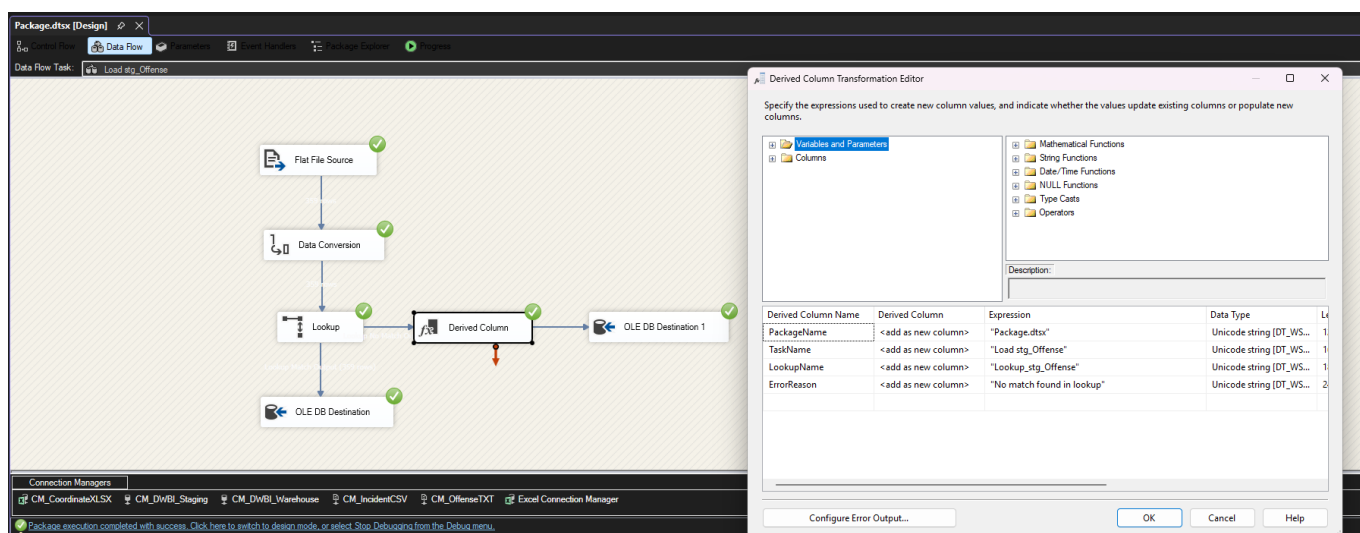


Figure 33: SSIS Derived Column Configuration for Capturing Lookup Failure Details in the Offense Loading Process

The **Derived Column** transformation was used in the **Load_stg_Offense** data flow to add metadata and error-handling information to the data stream. As shown in the image above, new columns including **PackageName**, **TaskName**, **LookupName**, and **ErrorReason** were created using fixed expressions. This step improves traceability in the ETL process by identifying the package and task involved, while also capturing the reason for rejected or unmatched records. Such metadata is useful for debugging, auditing, and maintaining the reliability of the ETL package.

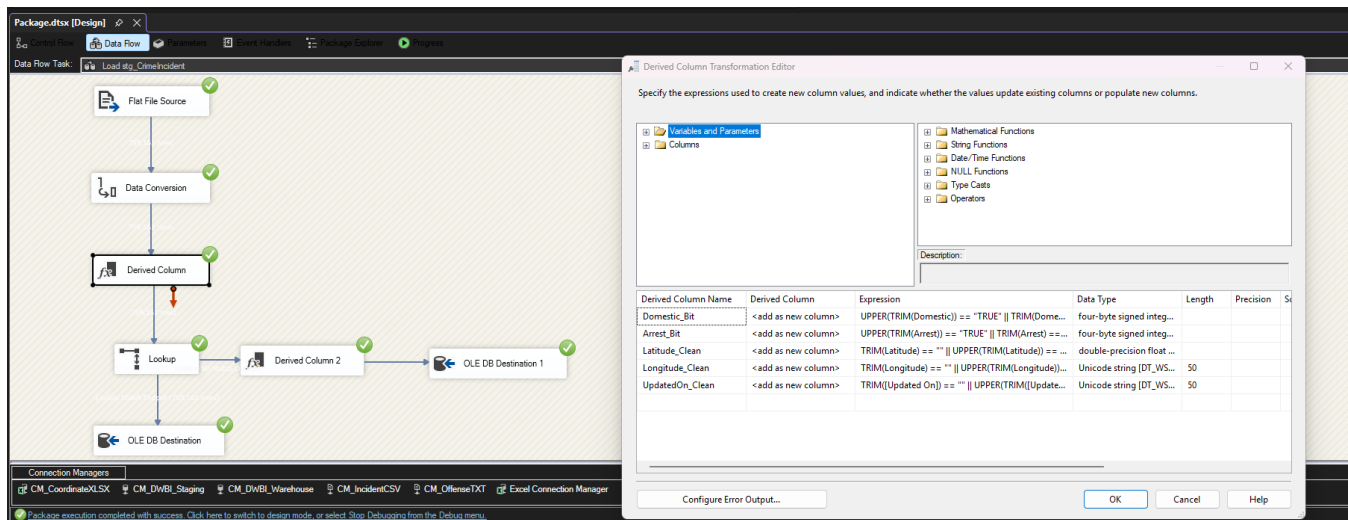


Figure 34: Configuration of the Derived Column Transformation in SSIS for Standardizing CrimeIncident Source Attributes

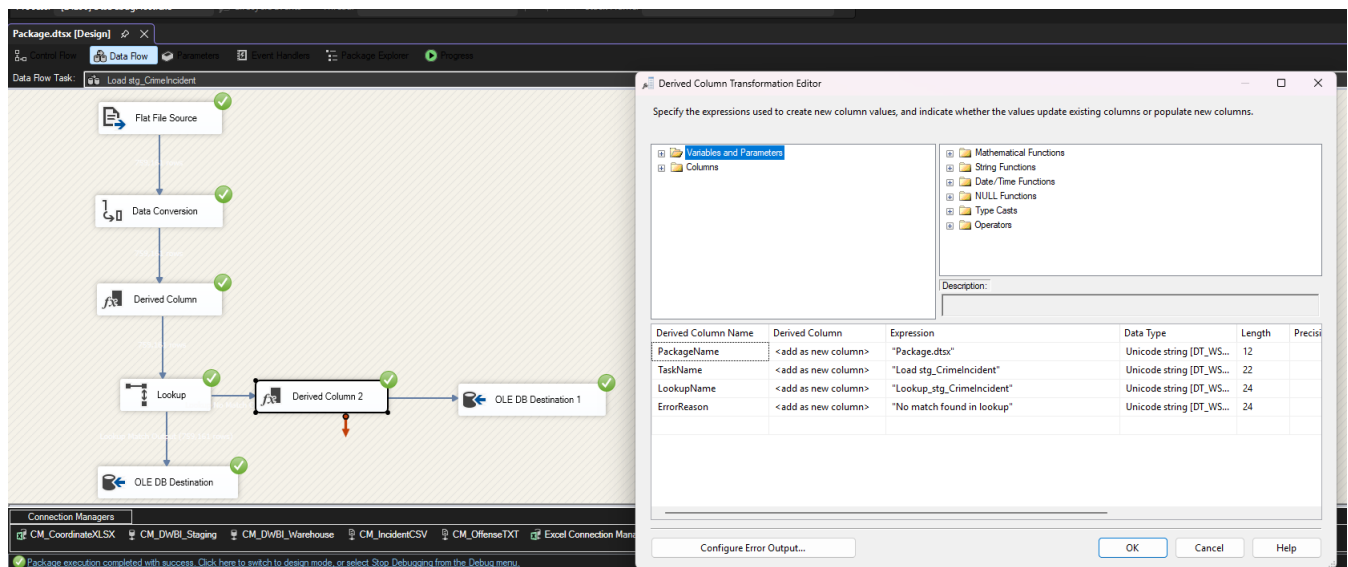


Figure 35: SSIS Derived Column Configuration for Capturing Lookup Failure Details in the CrimeIncident Loading Process

The **Derived Column** transformation was used twice in the **Load_stg_CrimeIncident** data flow. In the first transformation, source values were cleaned and standardized by creating derived columns such as **Domestic_Bit**, **Arrest_Bit**, **Latitude_Clean**, **Longitude_Clean**, and **UpdatedOn_Clean**. This ensured that text-based or incomplete source values were transformed into forms suitable for downstream processing. In the second transformation, metadata columns including **PackageName**, **TaskName**, **LookupName**, and **ErrorReason** were added to improve traceability and error handling for lookup failures. Together, these transformations improved data quality, consistency, and maintainability of the ETL process.

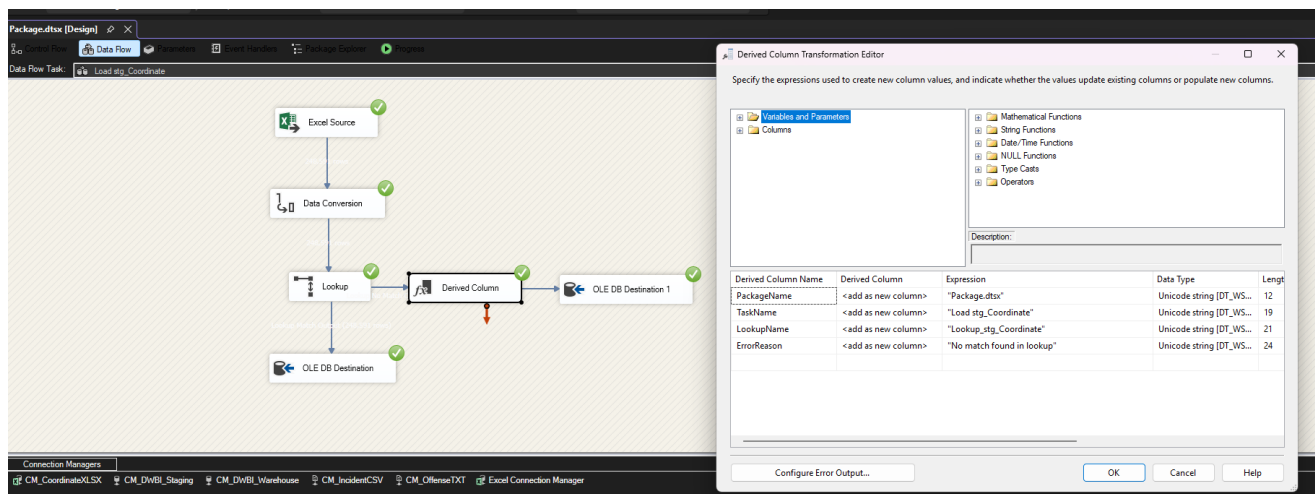


Figure 36: SSIS Derived Column Configuration for Capturing Lookup Failure Details in the Coordinate Loading Process

The **Derived Column** transformation was used in the **Load_stg_Coordinate** data flow to add metadata and error-handling information to the data stream. As shown in the image above, new columns including **PackageName**, **TaskName**, **LookupName**, and **ErrorReason** were created using fixed expressions. This supports ETL monitoring by identifying the package and task where processing occurred, while also recording the reason for unmatched lookup records. As a result, the transformation improves debugging, auditing, and the overall maintainability of the ETL package.

5.3.4 Error Handling

Error handling was explicitly built into the SSIS package to improve ETL reliability. Instead of allowing invalid rows to terminate the package, problematic rows were redirected into dedicated staging error tables. Conversion-related issues were captured in **ETL_Error_Conversion**, while lookup no-match rows were stored in **ETL_Error_Lookup_NoMatch**. This allowed the ETL process to complete successfully while preserving rejected records for later investigation. In addition, the error-handling design supports clearer troubleshooting because different error categories are isolated into separate tables rather than mixed into a single log structure.

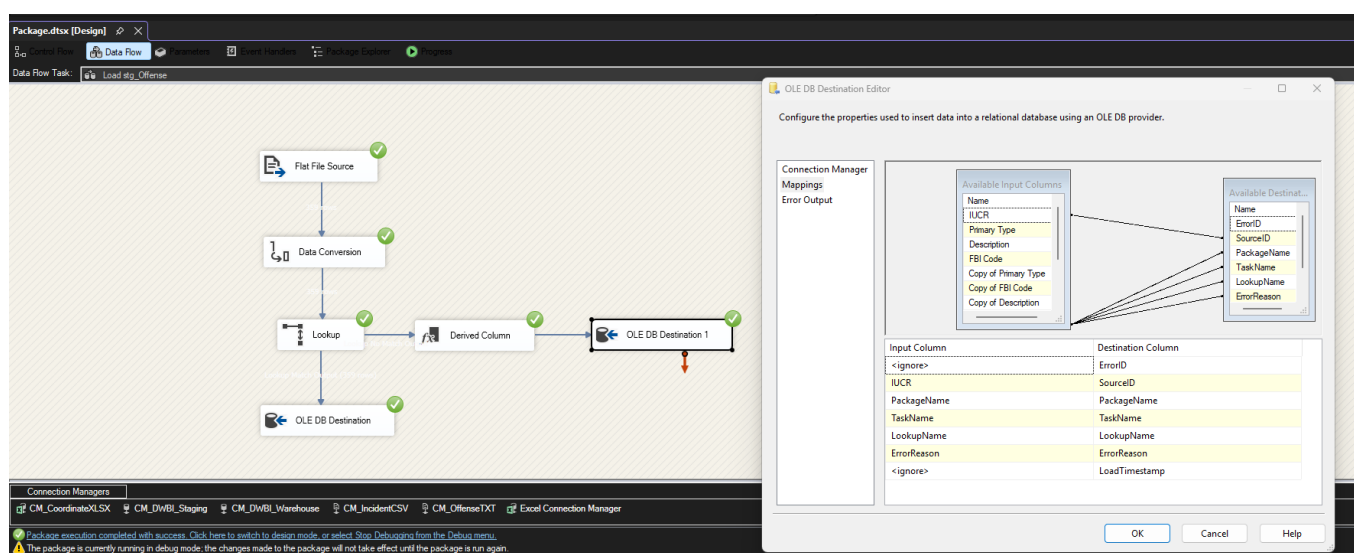


Figure 37: SSIS OLE DB Destination Configuration for Storing Lookup Failure Records from the Offense Loading Process

This image above shows the error-handling implementation in the **Load_stg_Offense** data flow. Records that fail the lookup are redirected through a separate error path, where metadata such as **PackageName**, **TaskName**, **LookupName**, and **ErrorReason** is added before loading the row into an error table through **OLE DB Destination**. This ensures that unmatched records are captured for auditing, debugging, and later correction instead of being lost.

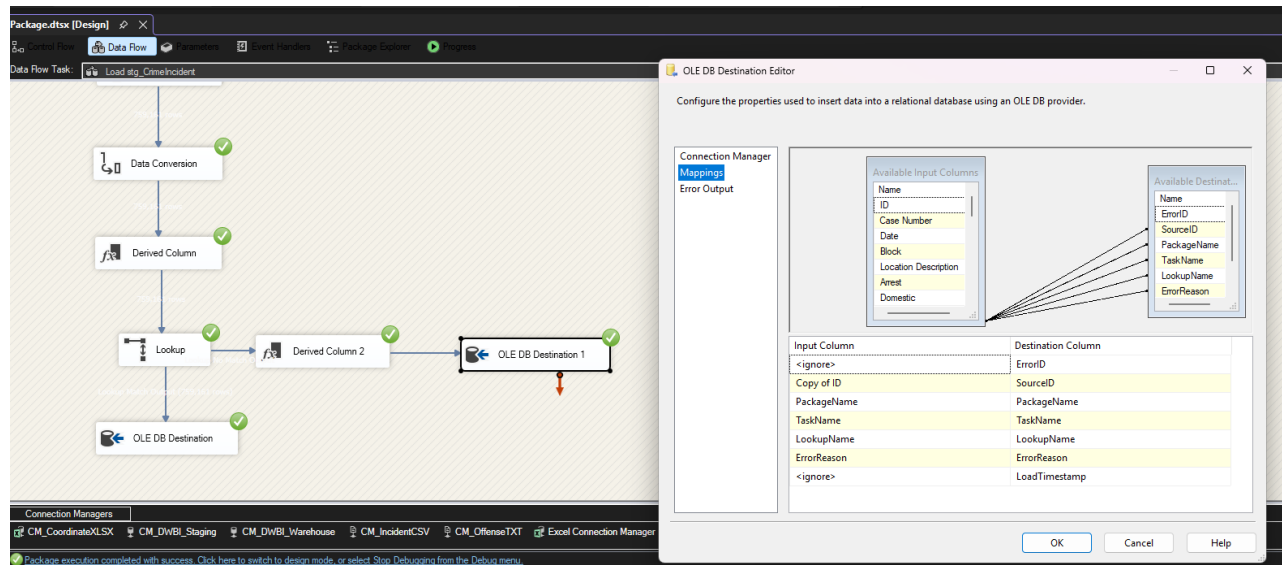


Figure 38: SSIS OLE DB Destination Configuration for Storing Lookup Failure Records from the CrimeIncident Loading Process

Above image shows the error-handling process in the **Load_stg_CrimeIncident** data flow. Records that fail the lookup are redirected to a separate path, where metadata such as **PackageName**, **TaskName**, **LookupName**, and **ErrorReason** is added through a Derived Column transformation. These rows are then loaded into an error table using **OLE DB Destination**, allowing failed records to be tracked, audited, and corrected later.

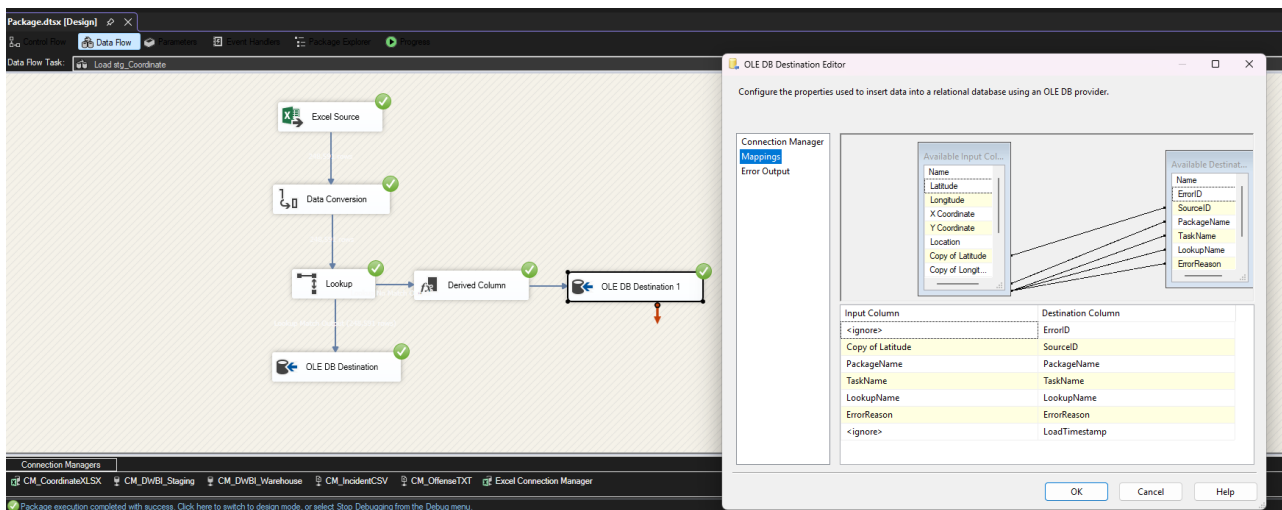


Figure 39: SSIS OLE DB Destination Configuration for Storing Lookup Failure Records from the Coordinate Loading Process

This image also shows the error-handling implementation in the **Load_stg_Coordinate** data flow. Records that fail the lookup are redirected through a separate error path, where metadata such as **PackageName**, **TaskName**, **LookupName**, and **ErrorReason** is added using a Derived Column transformation. The rejected rows are then loaded into an error table through **OLE DB Destination**, ensuring that failed records are preserved for auditing, debugging, and later correction.

Validation queries:

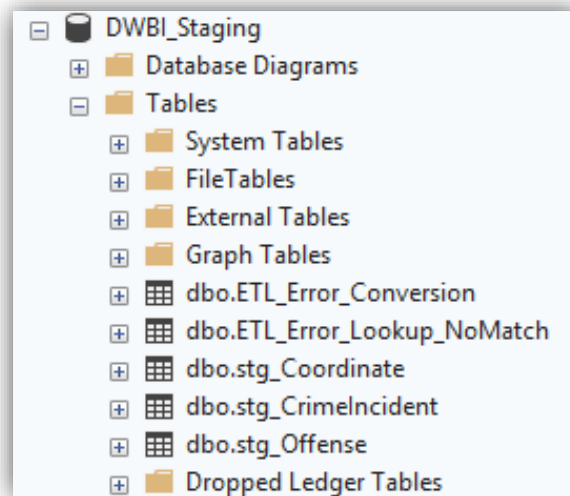


Figure 40: Schema View of the DWBI_Staging Database Showing Staging and ETL Error Tables

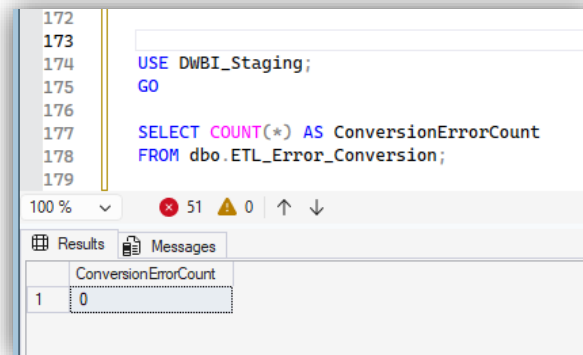


Figure 41: Verification of the Absence of Data Conversion Errors in ETL

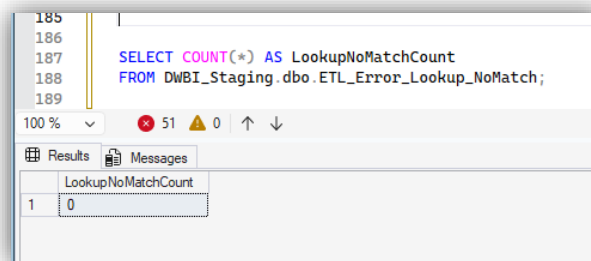


Figure 42: Verification of the Absence of Lookup No-Match Errors in ETL

These queries confirm that rejected rows were captured in dedicated error tables instead of being lost or causing package-level failure. This demonstrates that the ETL implementation includes practical row-level error handling.

5.4 Staging Table Loads

The first phase of the ETL process loads source data into staging tables. Staging tables act as an intermediate layer between the raw sources and the dimensional warehouse. This improves maintainability, simplifies transformation logic, and separates raw ingestion from dimensional processing.

The **Load_stg_Offense** data flow extracts offense data from a flat file and loads it into the staging database. The flow begins with a **Flat File Source**, after which a **Data Conversion** transformation is used to align incoming source columns with the required SQL Server data types. A **Lookup** transformation is then used to compare incoming rows against existing records, helping identify already-known values or duplicates. A **Derived Column** transformation may be used to prepare metadata or standardized attribute values before the rows are written to the destination. Valid rows are loaded into the staging table, while invalid or unmatched rows can be redirected to the error-handling path.

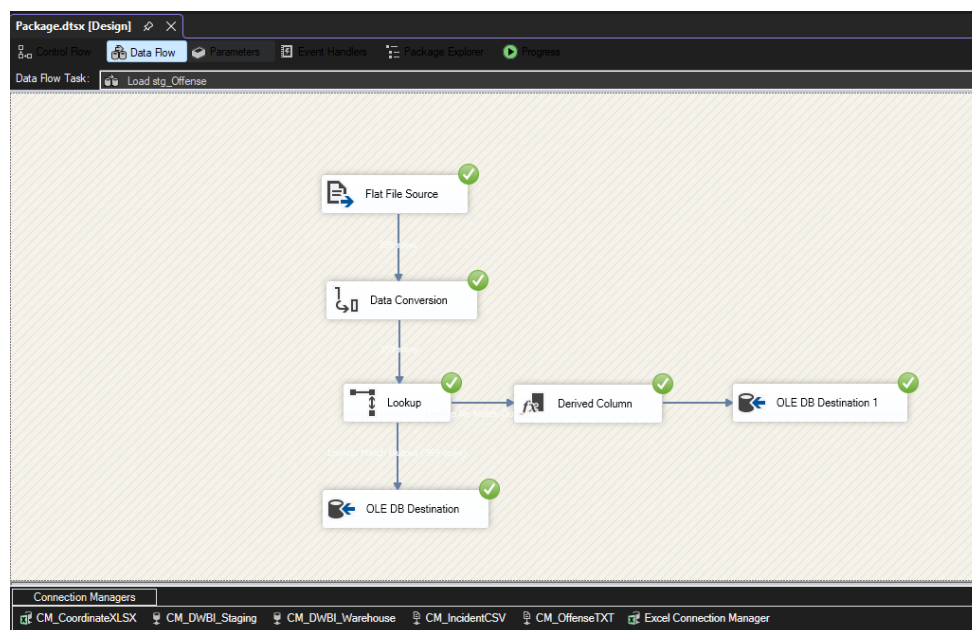


Figure 43: SSIS Execution View of the Data Flow Used to Transform and Load Offense Source Data

Staging steps:

- **Flat File Source** reads offense data from the source text file.
- **Data Conversion** aligns source columns with SQL Server-compatible data types.
- **Lookup** checks whether the incoming records already match reference data.
- **Derived Column** adds metadata or error-tracking values.
- **OLE DB Destination** loads valid rows into the staging table.

- Rejected rows are redirected through a separate **error-handling path**.

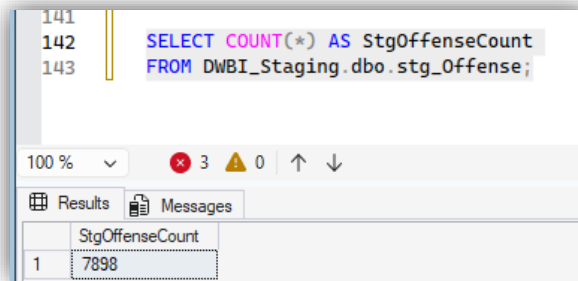


Figure 44: Verification of Record Count in the Offense Staging Table

This query confirms that the offense staging table was populated successfully. The row count provides evidence that the flat-file extraction and staging load completed as expected.

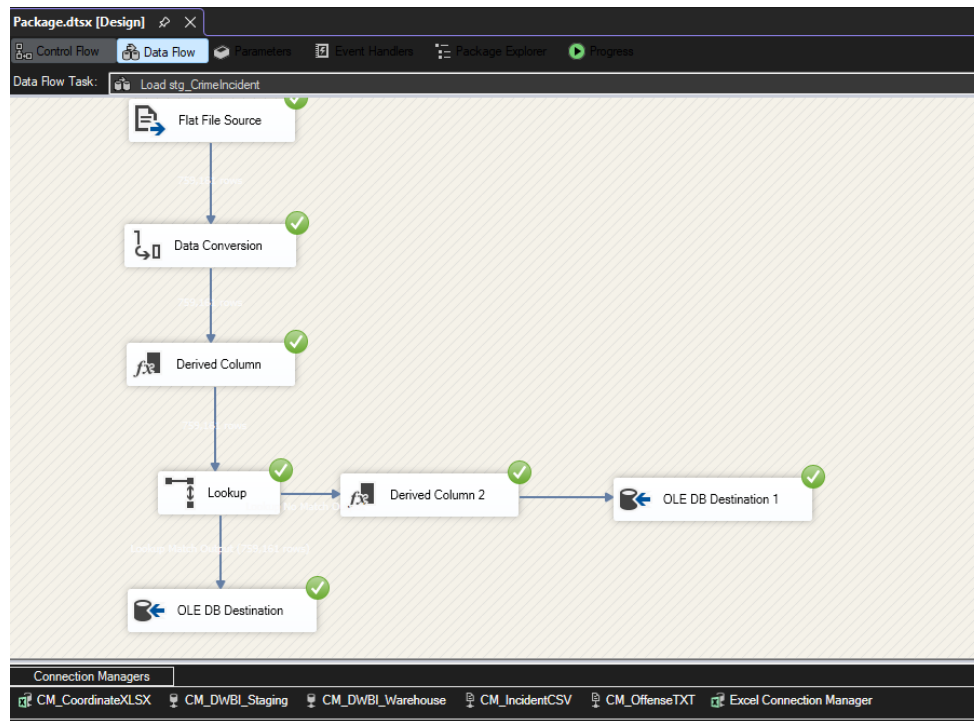


Figure 45: SSIS Execution View of the Data Flow Used to Transform and Load CrimeIncident Source Data

The **Load_stg_CrimeIncident** data flow loads crime incident data into the staging table through the following steps:

- Flat File Source** reads the crime incident data from the source file.
- Data Conversion** converts source columns into SSIS/SQL Server compatible data types.
- The first **Derived Column** transformation cleans and standardizes selected values before lookup.
- Lookup** checks incoming rows against reference data to identify matching

and non-matching records.

- The second **Derived Column** transformation adds metadata and error-related information where required.
- **OLE DB Destination 1** loads valid rows into the **crime incident staging table**.
- The lower **OLE DB Destination** handles redirected rows, storing rejected or unmatched records separately for error tracking.

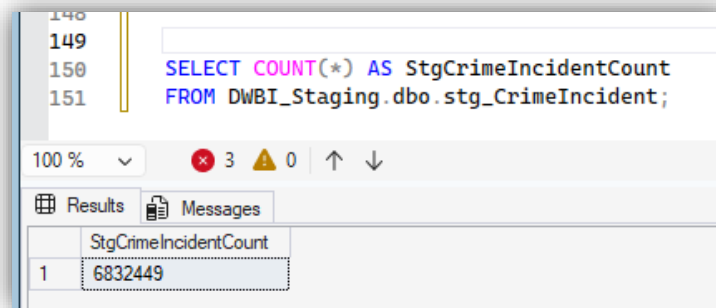


Figure 46: Verification of Record Count in the CrimeIncident Staging Table

The result confirms that crime incident data was successfully extracted, transformed, and loaded into the staging environment. This staging table becomes the main source for later dimension and fact loading.

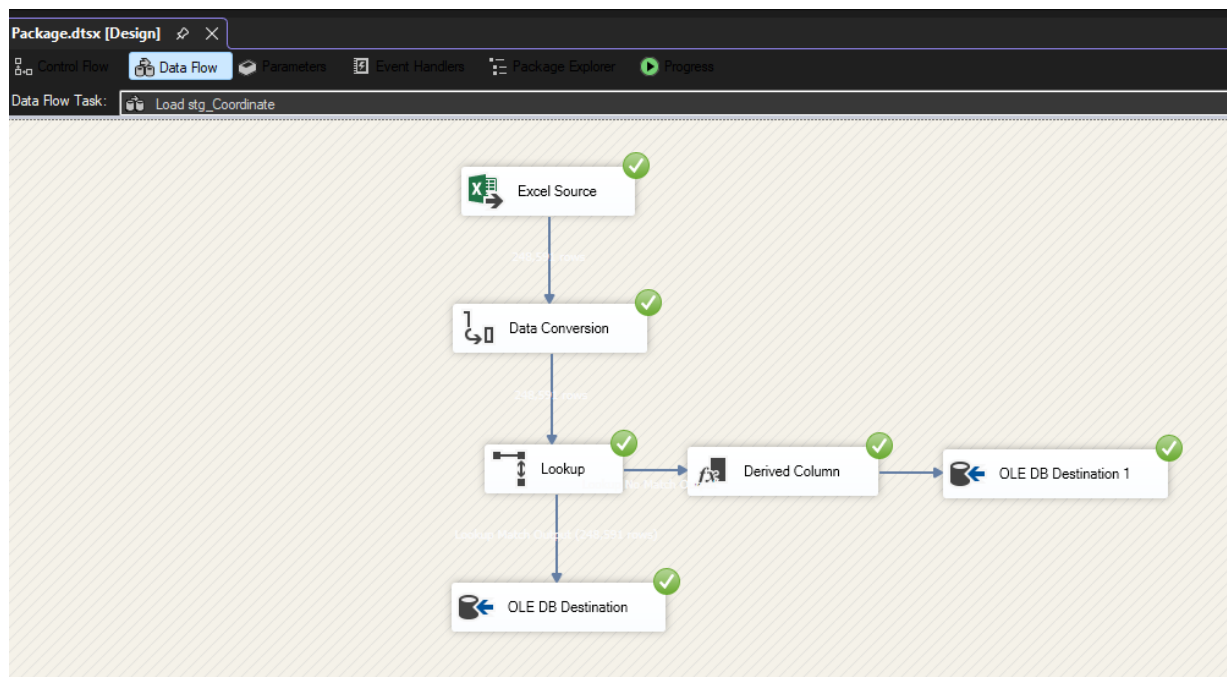


Figure 47: SSIS Execution View of the Data Flow Used to Transform and Load Coordinate Source Data

The **Load_stg_Coordinate** data flow loads coordinate data into the staging table through the following steps:

- **Excel Source** reads coordinate data from the Excel file.
- **Data Conversion** converts source columns into SSIS/SQL Server-compatible data types.
- **Lookup** checks incoming coordinate records against reference data to identify matching and non-matching rows.
- **Derived Column** adds metadata and error-related information where required.
- **OLE DB Destination 1** loads valid rows into the **coordinate staging table**.
- The lower **OLE DB Destination** stores redirected or unmatched rows separately for **error handling** and later review.

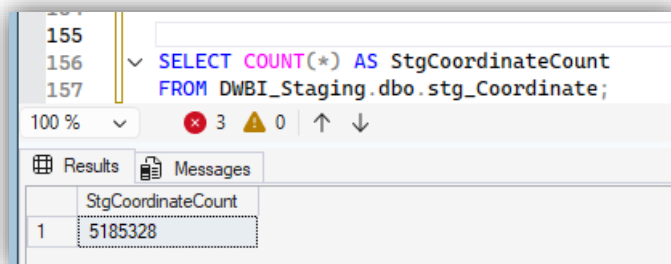


Figure 48: Verification of Record Count in the Coordinate Staging Table

This query validates that data from the Excel source was successfully staged in SQL Server. It also strengthens the evidence that multiple source types were used in the ETL solution.

5.5 Dimension Loads

After completing the staging loads, the ETL process populates the dimension tables. Loading the dimensions before the fact table is essential because the fact table requires valid surrogate keys from these dimensions. In this implementation, dimension loading takes place after data cleansing, transformation, and deduplication have already been handled in the staging phase. This ensures that the dimensional layer contains clean, structured, and analysis-ready data.

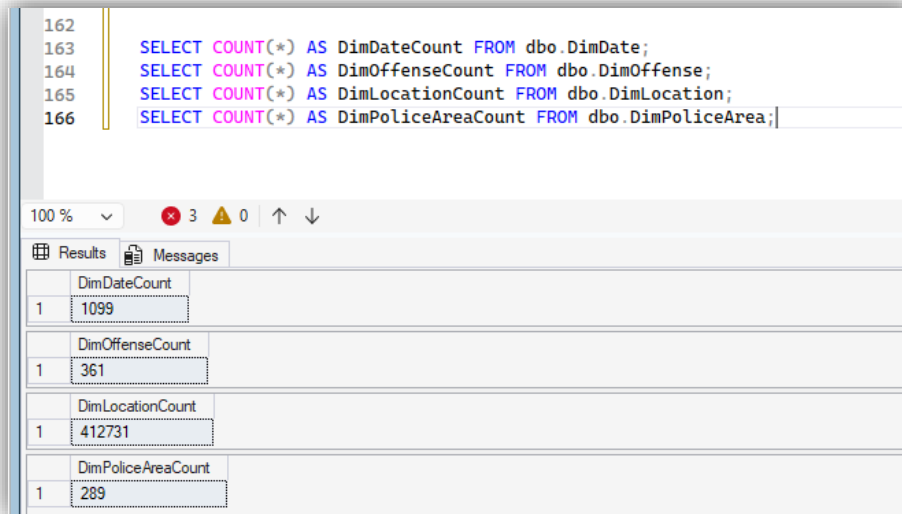


Figure 49: SQL-Based Verification of Record Counts in the Implemented Dimension Tables

These queries confirm that the dimension tables were successfully loaded before fact loading. Showing these counts is important because it proves that the ETL execution order was correct and that the dimensional structures required by the fact table are available.

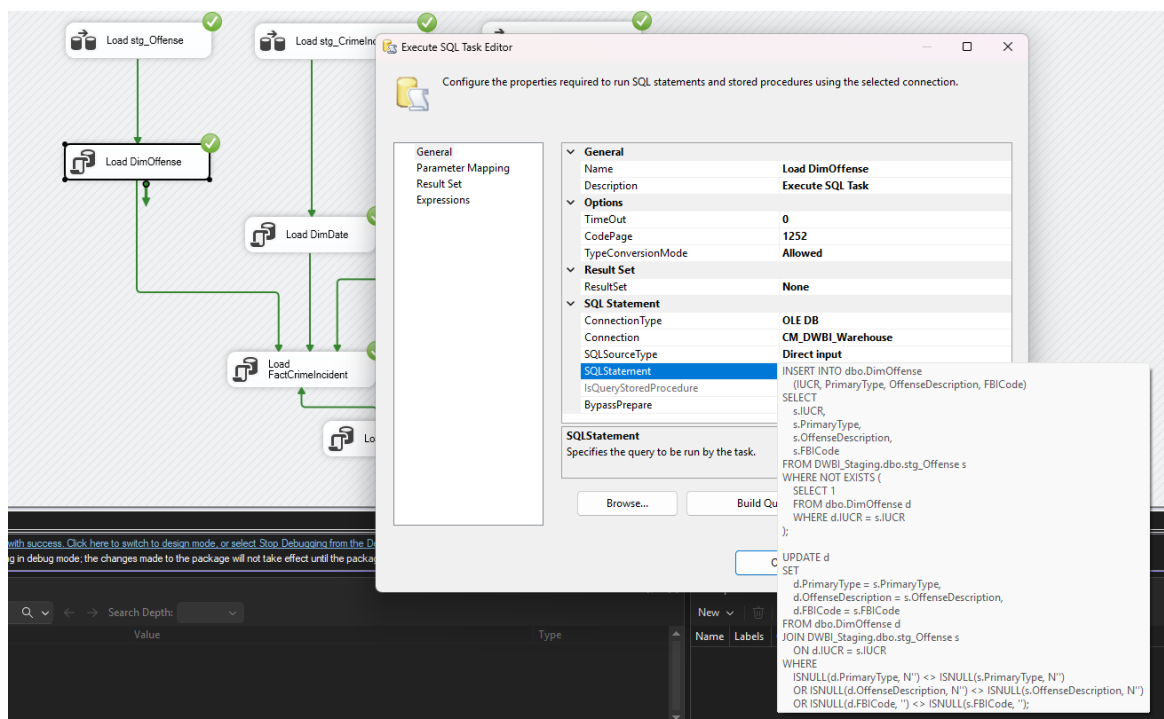


Figure 50: SSIS Execute SQL Task Configuration for Inserting and Updating Records in the DimOffense Table

This above image shows the **Execute SQL Task** responsible for loading **DimOffense** from the staging table. The task uses SQL statements to insert new records and update existing dimension rows based on the staged offense data. This step ensures that the dimension table remains consistent with the latest processed source data.

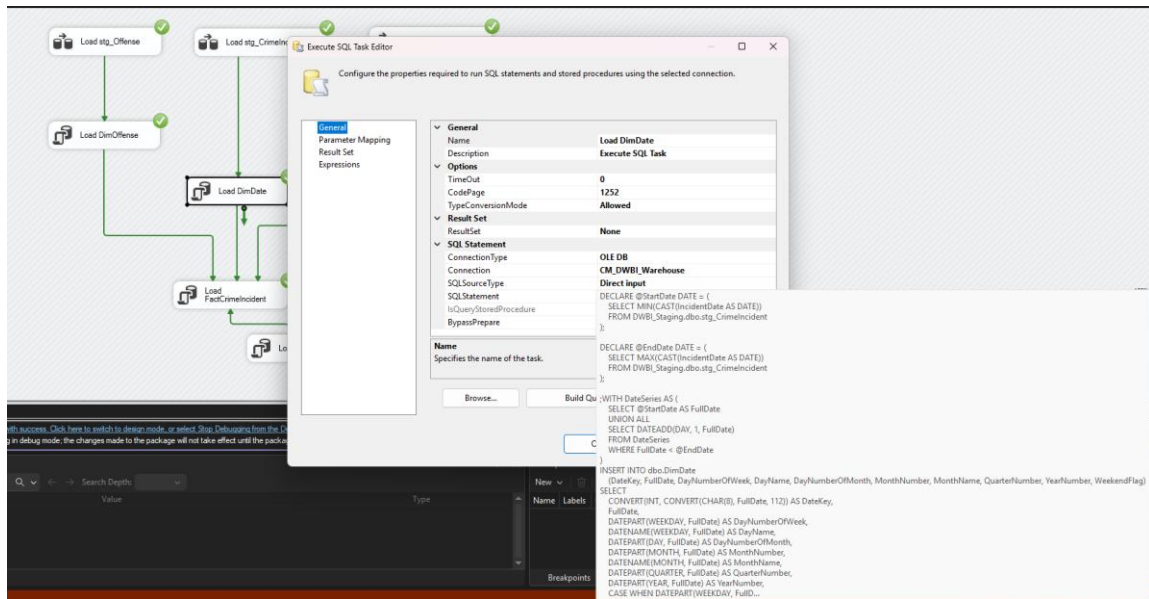


Figure 51: SSIS Execute SQL Task Configuration for Populating the DimDate Table from Crime Incident Dates

The **DimDate** dimension was loaded using an **Execute SQL Task**. The SQL script first determines the minimum and maximum incident dates from the staging table, then generates a continuous series of dates between those boundaries. For each generated date, descriptive calendar attributes such as **DateKey**, **DayNumberOfWeek**, **DayName**, **DayNumberOfMonth**, **MonthNumber**, **MonthName**, **QuarterNumber**, **YearNumber**, and **WeekendFlag** are derived and inserted into **dbo.DimDate**. This ensures that the date dimension contains a complete range of dates required for time-based analysis.

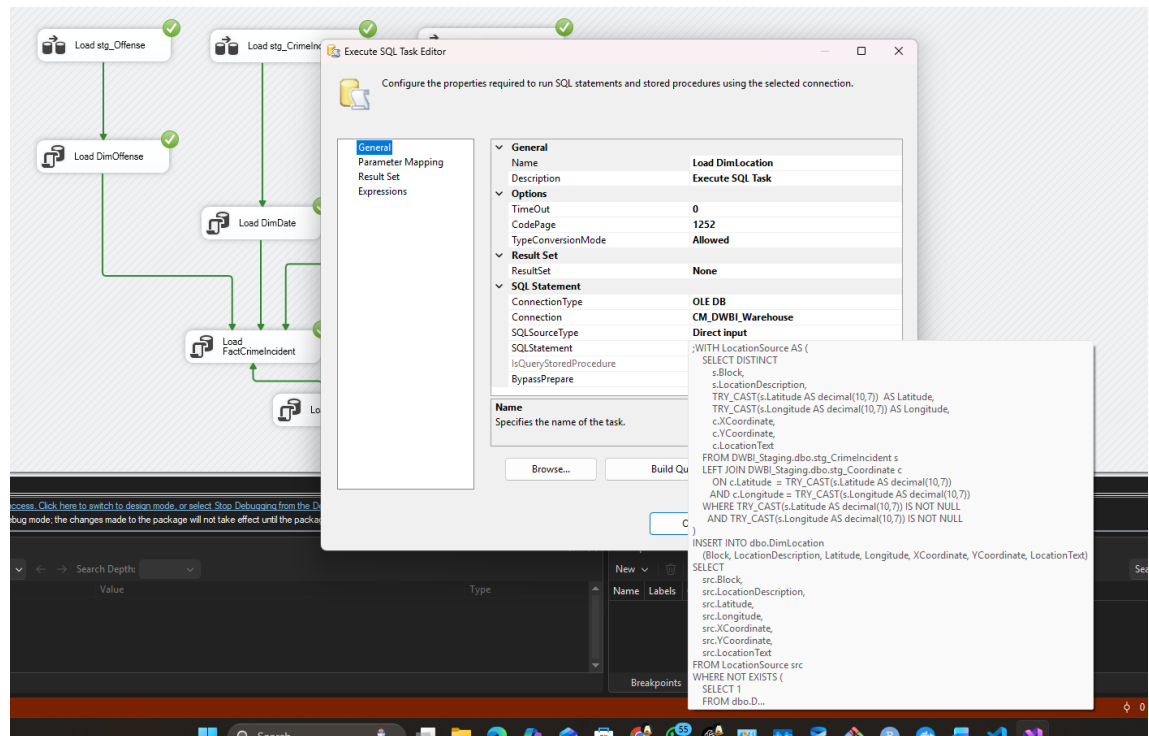


Figure 52: SSIS Execute SQL Task Configuration for Populating the DimLocation Table

The **DimLocation** dimension was loaded using an **Execute SQL Task**. The SQL logic first prepares a location source set by selecting distinct location-related attributes from the staging tables, including **Block**, **LocationDescription**, **Latitude**, **Longitude**, **XCoordinate**, **YCoordinate**, and **LocationText**. The script then inserts new location records into **dbo.DimLocation** where a matching dimension row does not already exist. This ensures that the location dimension contains unique location entries for later fact table loading.

5.6 Slowly Changing Dimension Implementation – DimPoliceArea

The **DimPoliceArea** dimension was implemented as a **Slowly Changing Dimension (SCD)** using set-based SQL inside an **Execute SQL Task**. In this design, the ETL first identifies the latest source row for each business key combination (**DistrictCode** + **BeatCode**). If the tracked attributes for the current row have changed, the previous row is expired by updating its current-row indicator and end date, and a new current row is inserted. This preserves historical changes while ensuring that only one version remains current for each business key.

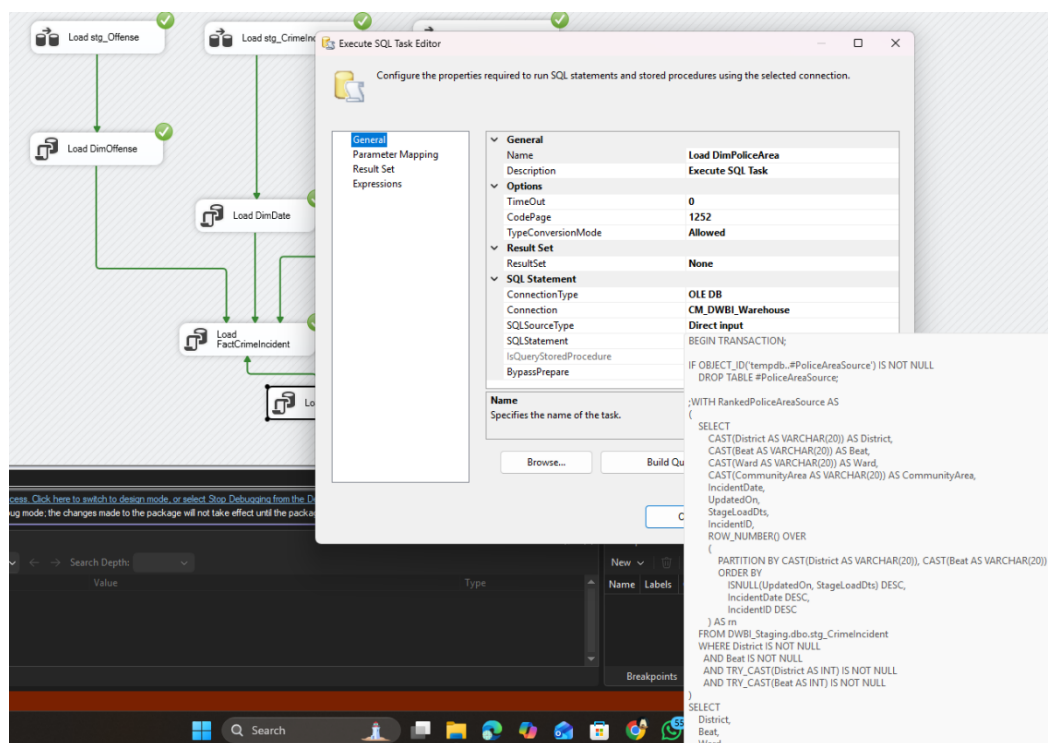


Figure 53: SSIS Execute SQL Task Configuration for Populating the DimPoliceArea Table

The **Load DimPoliceArea** task was implemented using an **Execute SQL Task** with set-based SQL. As shown in the image above, the script extracts police area records from the staging table, ranks source rows to identify the latest version for

each **District–Beat** business key, and then applies change-detection logic. Where attribute changes are found, the previous current row is expired and a new current row is inserted. This ensures that historical changes are retained while keeping a single active current record for each police area.

5.7 Fact Table Load – FactCrimeIncident

Once the staging and dimension tables are loaded, the final main ETL step is the loading of the fact table FactCrimeIncident. The fact ETL deduplicates source incidents using IncidentID, resolves surrogate keys by joining to the dimensions, calculates analytical measures, and inserts only those rows that do not already exist in the fact table. This step is central to the dimensional model because it creates the measurable event-level dataset that will later support reporting and analysis.

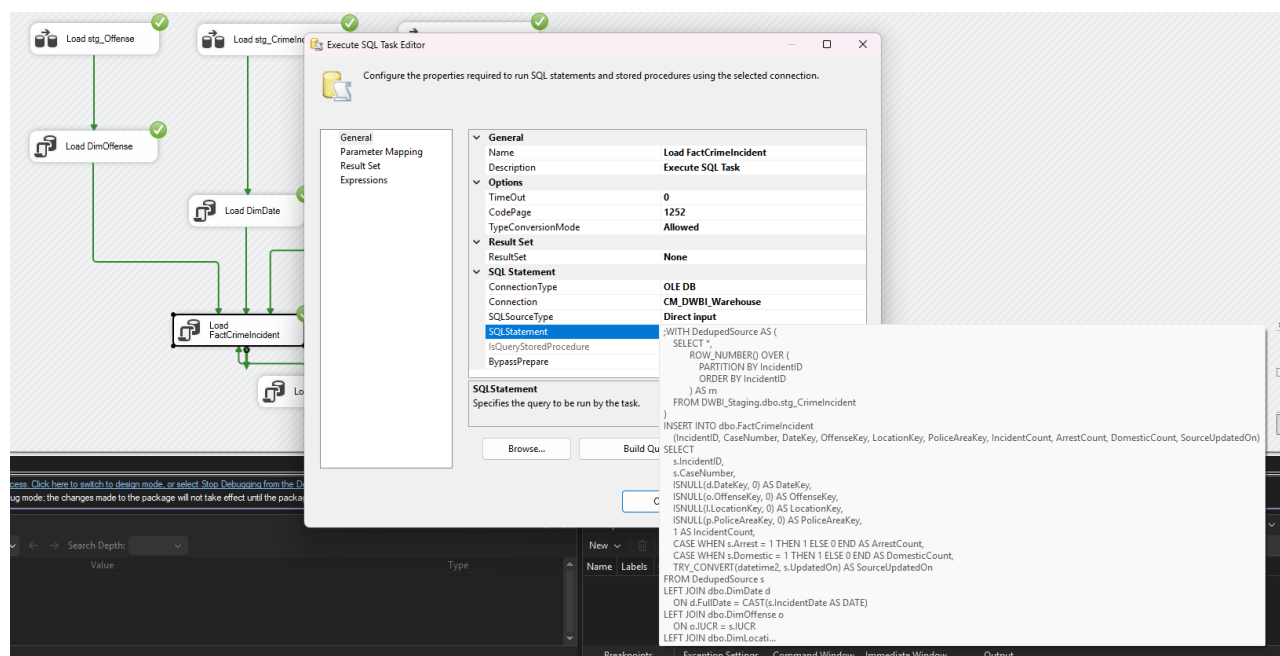


Figure 54: SSIS Execute SQL Task Configuration for Populating the FactCrimeIncident Table

The **FactCrimeIncident** fact table was loaded using an **Execute SQL Task**. The SQL script reads deduplicated crime incident data from the staging table and joins it with the relevant dimension tables to retrieve surrogate keys, including **DateKey**, **OffenseKey**, **LocationKey**, and **PoliceAreaKey**. The script then inserts the transformed records into **dbo.FactCrimeIncident** together with derived measures such as **IncidentCount**, **ArrestCount**, and **DomesticCount**. This ensures that the fact table is properly linked to the dimension tables and supports analytical querying.

5.8 Yearly Summary Aggregate Load

To support aggregate-based analysis, an additional SSIS data flow was implemented after loading the fact table. This data flow summarizes detailed fact records from FactCrimeIncident into yearly aggregate values and loads them into the summary table FactIncidentYearSummary in the warehouse. The process uses an OLE DB Source to extract fact rows joined with the date dimension, an Aggregate transformation to group records by YearNumber and calculate summary measures, and an OLE DB Destination to store the aggregated results in the warehouse.

In this implementation, the aggregate step calculates yearly totals such as TotalIncidents, TotalArrests, and TotalDomestic. This demonstrates that the warehouse not only stores detailed fact-level data, but also supports summarized analytical views that can be used for reporting and business intelligence.

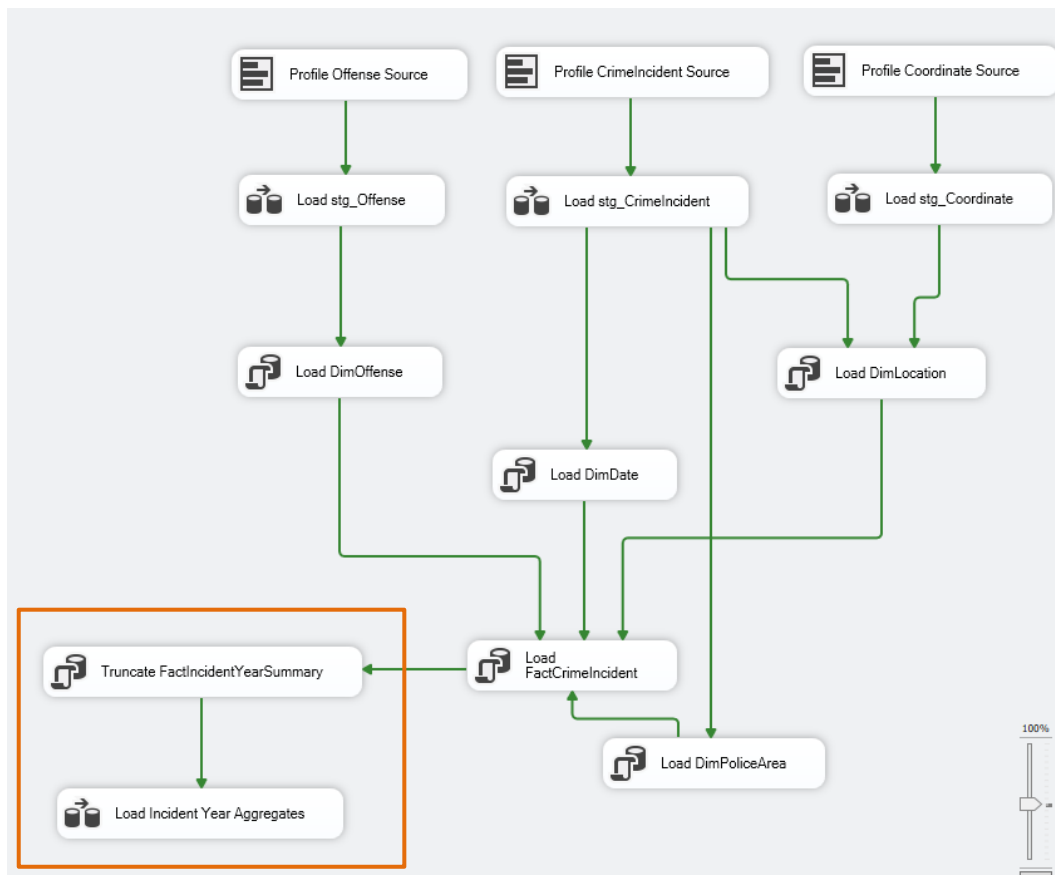


Figure 55: SSIS Control Flow for Populating the Fact and Dimension Tables and Generating the Incident Year Summary

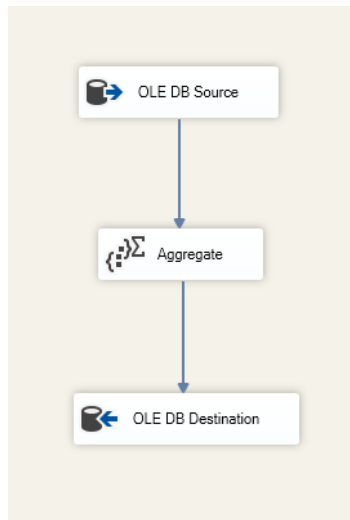


Figure 56: SSIS Data Flow Employing an Aggregate Transformation to Populate the Incident Year Summary Table

This SSIS data flow implements aggregate processing for the warehouse. Data is extracted from the warehouse through an OLE DB Source, grouped and summarized using the Aggregate transformation, and then loaded into a summary table through an OLE DB Destination. In this implementation, the flow is used to calculate yearly summary measures such as total incidents, arrests, and domestic cases.

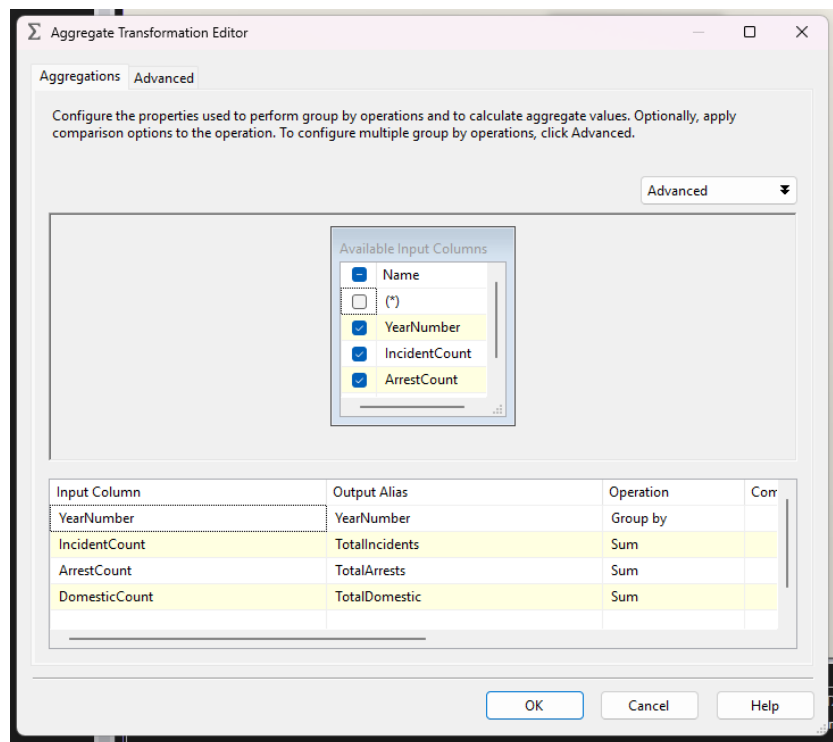


Figure 57: SSIS Aggregate Transformation Editor for Yearly Incident, Arrest, and Domestic Totals

The Aggregate transformation groups fact records by YearNumber and calculates summary measures for each year. In this implementation, IncidentCount, ArrestCount, and DomesticCount are summed to produce TotalIncidents, TotalArrests, and TotalDomestic. This step is used to generate yearly aggregate data for reporting and analysis

Task 6: ETL development – Accumulating fact tables

6.0 Extending the fact table

The fact table **FactCrimeIncident** was extended with the additional columns **accm_txn_create_time**, **accm_txn_complete_time**, and **txn_process_time_hours** to support the accumulating fact table requirement. The create timestamp records when the fact row was initially loaded, the completion timestamp stores the later completion time, and the processing-hours column stores the difference in hours between the two timestamps.

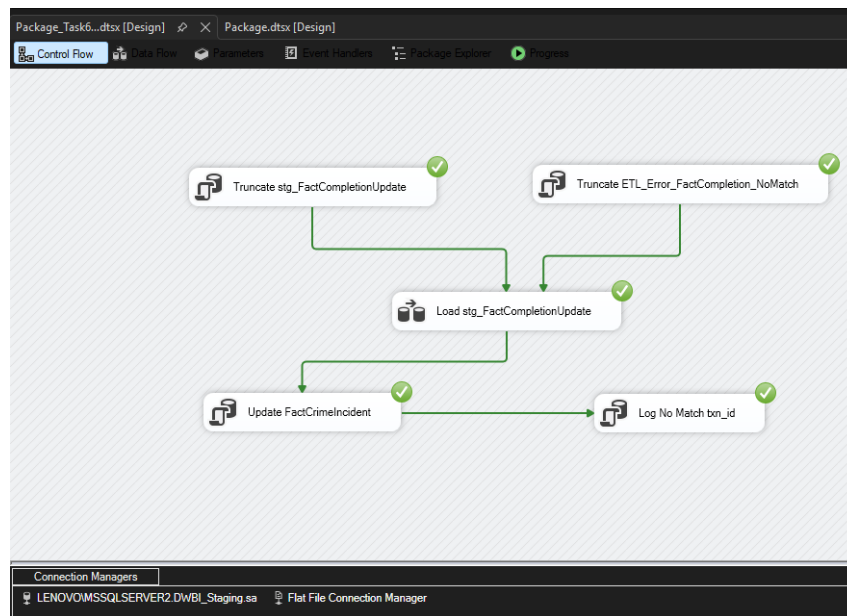


Figure 58: SSIS Control Flow for Incrementally Updating the FactCrimeIncident Table and Logging No-Match Records

6.1 Initial fact load

During the initial fact load, **accm_txn_create_time** was populated using the current system timestamp, while **accm_txn_complete_time** and **txn_process_time_hours** were initially kept NULL. This follows the accumulating fact concept where the transaction is created first and completed later.

The screenshot shows a SQL query window with the following query:

```
SELECT *
FROM DWBI_Staging.dbo.stg_FactCompletionUpdate;
```

The results pane shows the following data:

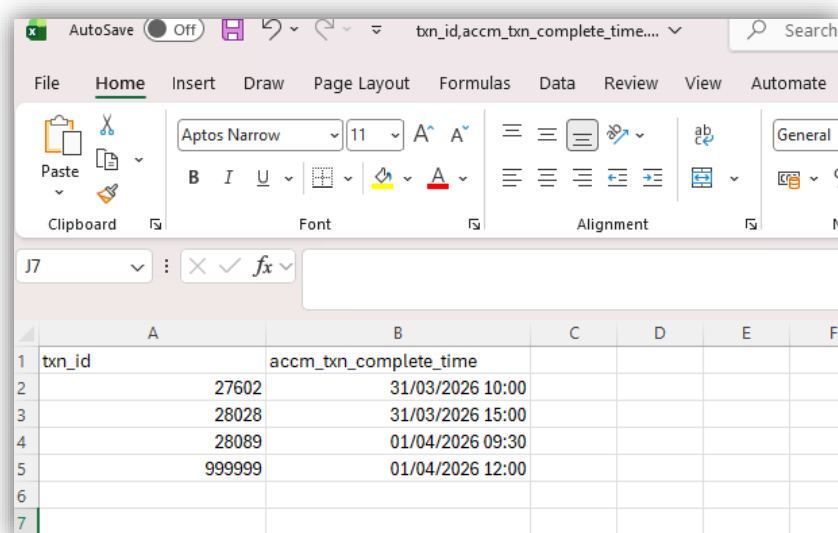
txn_id	accm_txn_complete_time
27602	2026-03-31 10:00:00.000
28028	2026-03-31 15:00:00.000
28089	2026-04-01 09:30:00.000
999999	2026-04-01 12:00:00.000

Figure 59: Sample Records from the stg_FactCompletionUpdate Table

The above figure shows the contents of **stg_FactCompletionUpdate** after the CSV file was loaded through the SSIS Data Flow. Four rows were loaded into the staging table, including three matching transaction IDs and one unmatched transaction ID (999999) used for error logging validation.

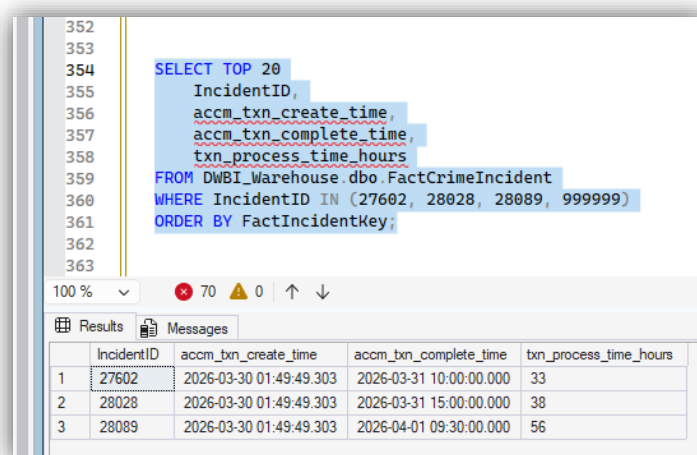
6.2 Separate completion dataset

A separate CSV file was prepared containing **txn_id** and **accm_txn_complete_time**. In this implementation, **txn_id** corresponds to IncidentID in FactCrimeIncident. This dataset was used as the source for updating the completion time in the fact table through a separate SSIS package.



txn_id	accm_txn_complete_time
27602	31/03/2026 10:00
28028	31/03/2026 15:00
28089	01/04/2026 09:30
999999	01/04/2026 12:00

Figure 60: Source Spreadsheet Containing Completion Update Records for Incremental Maintenance of the FactCrimeIncident Table



```

SELECT TOP 20
    IncidentID,
    accm_txn_create_time,
    accm_txn_complete_time,
    txn_process_time_hours
FROM DWBI_Warehouse.dbo.FactCrimeIncident
WHERE IncidentID IN (27602, 28028, 28089, 999999)
ORDER BY FactIncidentKey;

```

IncidentID	accm_txn_create_time	accm_txn_complete_time	txn_process_time_hours
27602	2026-03-30 01:49:49.303	2026-03-31 10:00:00.000	33
28028	2026-03-30 01:49:49.303	2026-03-31 15:00:00.000	38
28089	2026-03-30 01:49:49.303	2026-04-01 09:30:00.000	56

Figure 61: Verification of Completion Time Updates in the FactCrimeIncident Table

The above figure shows the updated rows in **FactCrimeIncident** after executing the SSIS package. The values for **accm_txn_complete_time** were successfully

updated for the matching transaction IDs **27602**, **28028**, and **28089**. The column **txn_process_time_hours** was also calculated for each updated row.

6.3 Separate SSIS package

A separate SSIS package named **Package_Task6_AccumulatingFact.dtsx** was developed for the accumulating fact update process. The package first truncates the staging and error tables, then loads the CSV file into the staging table, updates matching rows in the fact table, and finally logs unmatched transaction IDs into a no-match error table.

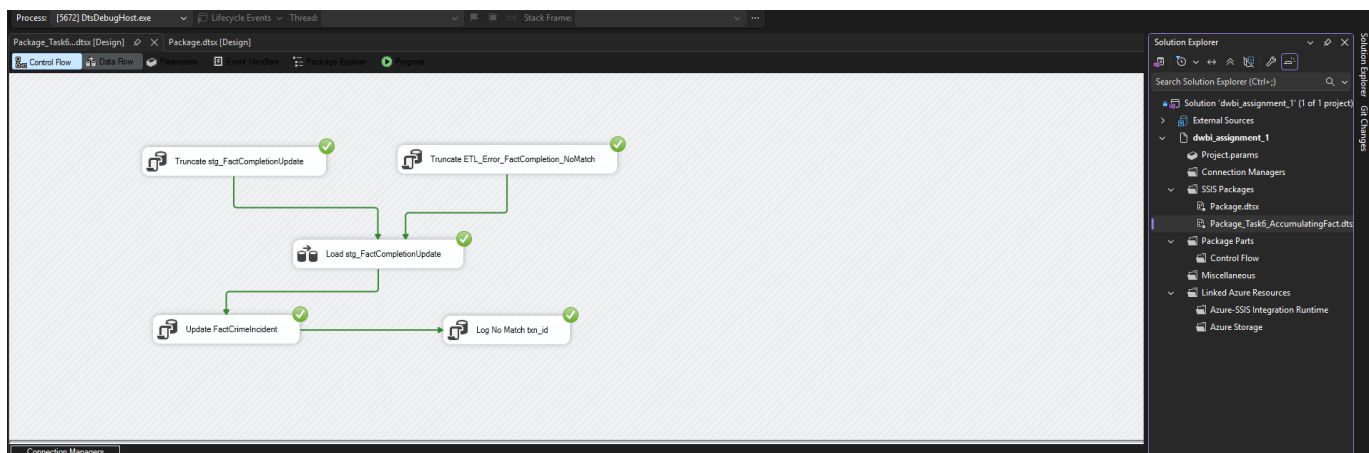


Figure 62: Implemented SSIS Package for Fact Table Completion Update Processing

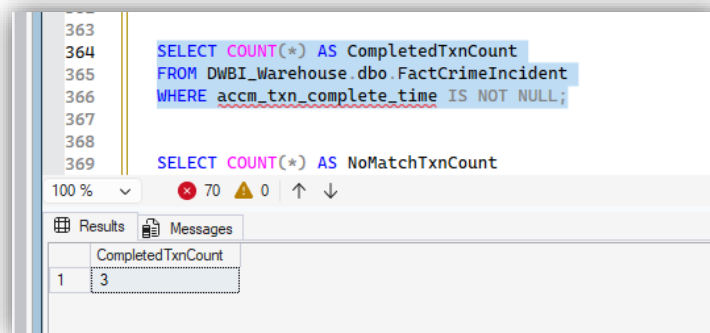


Figure 63: Verification of Updated Completion Time Records in the Fact Table

The figure above presents the validation query used to count completed transactions in the fact table. The result shows that **3** rows in **FactCrimeIncident** have non-null completion timestamps after the update process. This confirms that the matching records were updated successfully.

6.4 Update logic

The package loads the CSV file into **DWBI_Staging.dbo.stg_FactCompletionUpdate**. The update step matches **txn_id** from the staging table with **IncidentID** in **DWBI_Warehouse.dbo.FactCrimeIncident**. For matched rows, **accm_txn_complete_time** is updated and **txn_process_time_hours** is calculated using `DATEDIFF(HOUR, accm_txn_create_time, accm_txn_complete_time)`. Any unmatched transaction IDs are inserted into **DWBI_Staging.dbo.ETL_Error_FactCompletion_NoMatch** for auditing and troubleshooting.

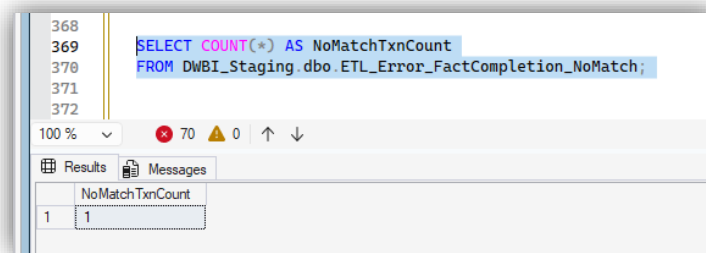


Figure 64: Verification of Updated Completion Time Records in the Fact Table

The above figure shows the validation query for counting unmatched transaction IDs stored in the error table. The result is **1**, confirming that one row from the CSV file did not have a matching **IncidentID** in the fact table and was correctly redirected to the error table.

6.5 Package configuration

The separate SSIS package for the accumulating fact update was configured using a sequence of Execute SQL Tasks and a Data Flow Task. The package first clears the staging and error tables, then loads the CSV completion dataset into the staging table, updates matching records in the fact table, and finally logs unmatched transaction IDs into a separate error table. This structure supports the assignment requirement for a separate ETL package that updates the accumulating fact table and recalculates processing time.

Execute SQL Task for truncating the staging table

This figure shows the configuration of the Execute SQL Task used to truncate **DWBI_Staging.dbo.stg_FactCompletionUpdate** before loading new CSV data. This ensures that old staging data is removed before each execution of the package.

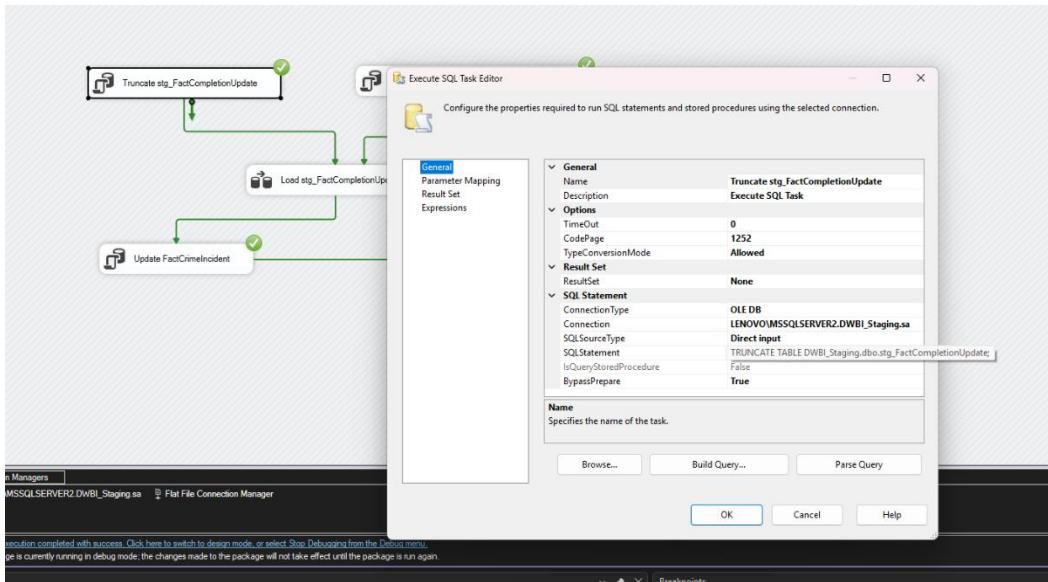


Figure 65: Execute SQL Task Configuration for Truncating the Fact Completion Update Staging Table

Execute SQL Task for truncating the no-match error table

This figure shows the configuration of the Execute SQL Task used to clear DWBI_Staging.dbo.ETL_Error_FactCompletion_NoMatch before each package run. This step ensures that only the unmatched records from the current execution are stored in the error table.

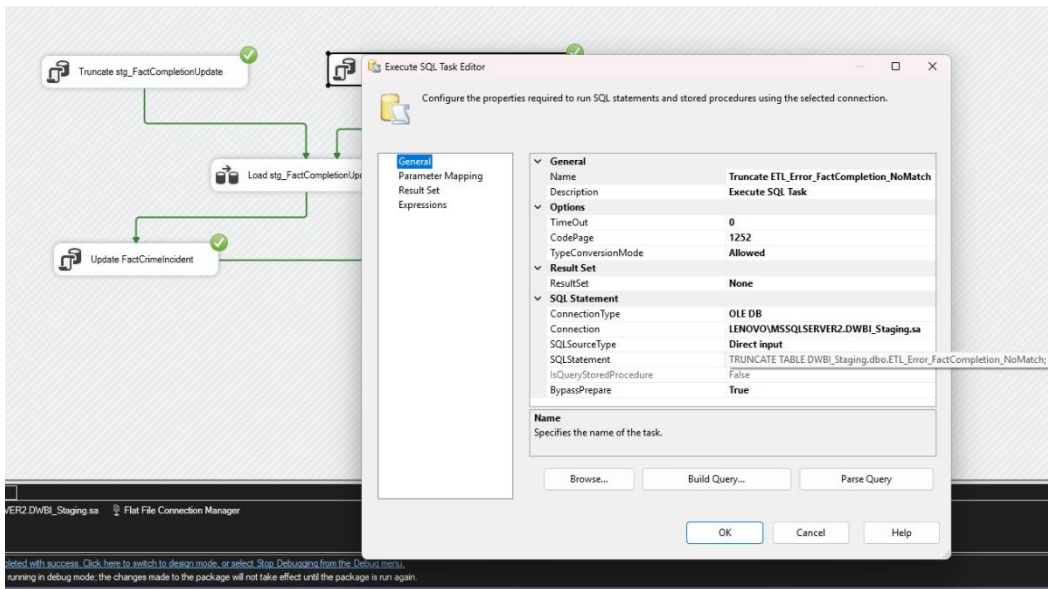


Figure 66: Execute SQL Task Configuration for Truncating the Fact Completion No-Match Error Table

Execute SQL Task for updating FactCrimeIncident

This figure shows the SQL configuration used to update the accumulating fact table. The task matches `txn_id` from `stg_FactCompletionUpdate` with `IncidentID` in `FactCrimeIncident`, updates `accm_txn_complete_time`, and calculates `txn_process_time_hours` using the difference between the completion and

creation timestamps. This is the core update step of the accumulating fact process.

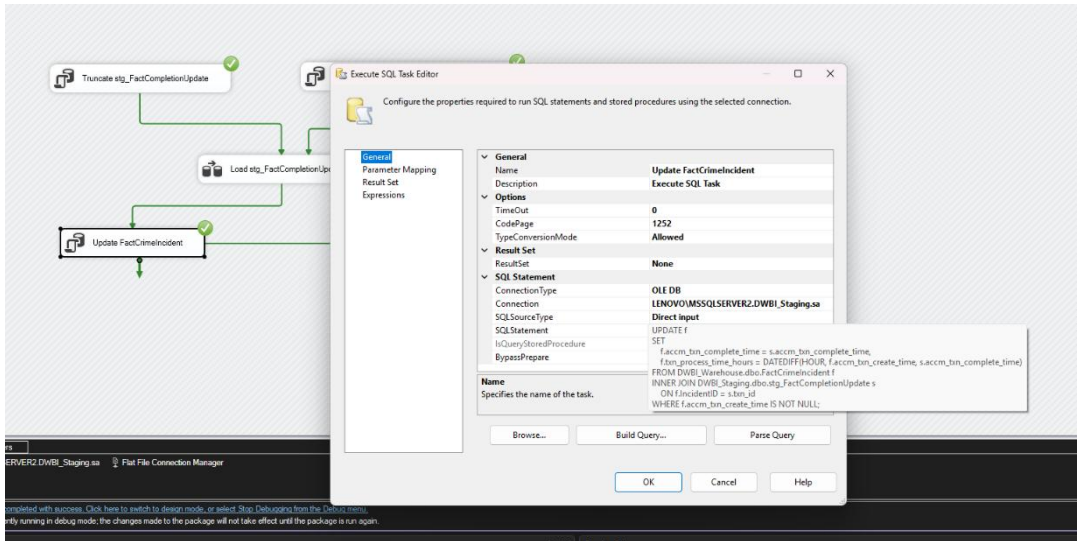


Figure 67: Execute SQL Task Configuration for Updating Completion Attributes in the FactCrimeIncident Table

Execute SQL Task for logging unmatched transaction IDs

This figure shows the SQL configuration used to insert unmatched rows into DWBI_Staging.dbo.ETL_Error_FactCompletion_NoMatch. Any txn_id from the staging table that does not have a matching IncidentID in the fact table is redirected into this table for auditing and troubleshooting. The assignment specifically expects handling of missing transaction IDs, and this step fulfills that requirement.

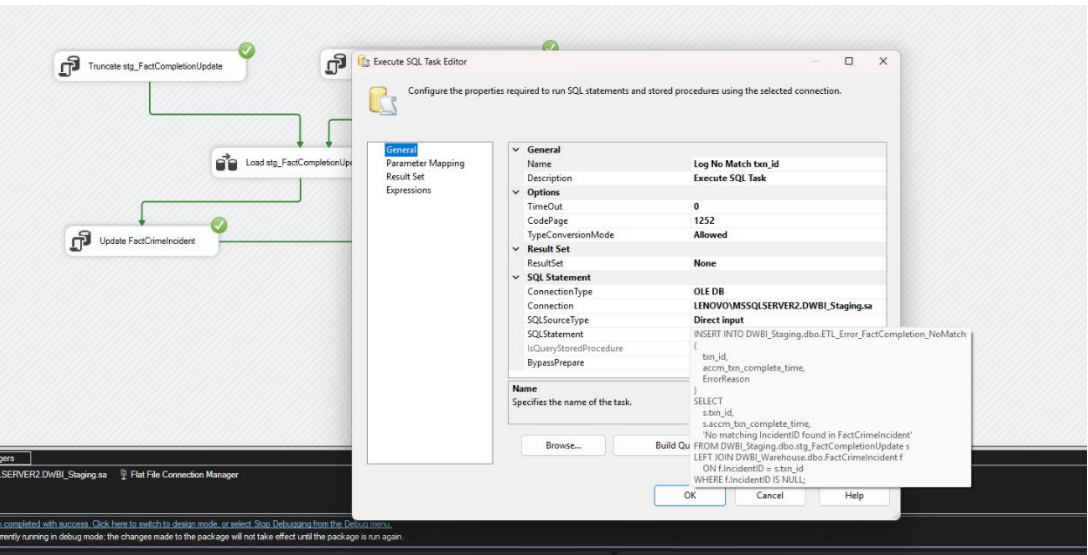


Figure 68: Execute SQL Task Configuration for Logging Unmatched Transaction IDs During Fact Completion Updates

Data Flow for loading the staging table

This figure shows the Data Flow used to load the CSV file into stg_FactCompletionUpdate. The flow consists of a Flat File Source, a Data Conversion transformation, and an OLE DB Destination. The Flat File Source reads the CSV file, the Data Conversion step converts the columns into the correct SQL-compatible data types, and the OLE DB Destination loads the transformed rows into the staging table.

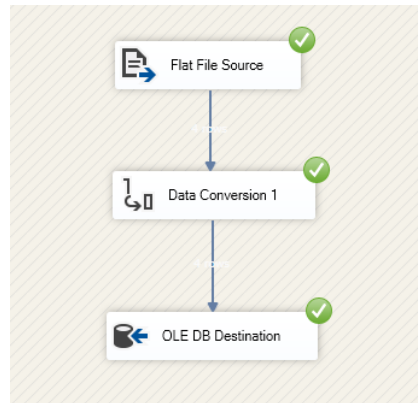


Figure 69: SSIS Data Flow for Loading the Fact Completion Update Staging Table

Data Conversion configuration

This figure shows the Data Conversion transformation used in the Data Flow. The txn_id column was converted to a four-byte signed integer to match the SQL INT data type, while accm_txn_complete_time was converted to a database timestamp type to match the SQL DATETIME field in the staging table. This conversion was necessary to ensure successful insertion into the staging table and correct downstream processing.

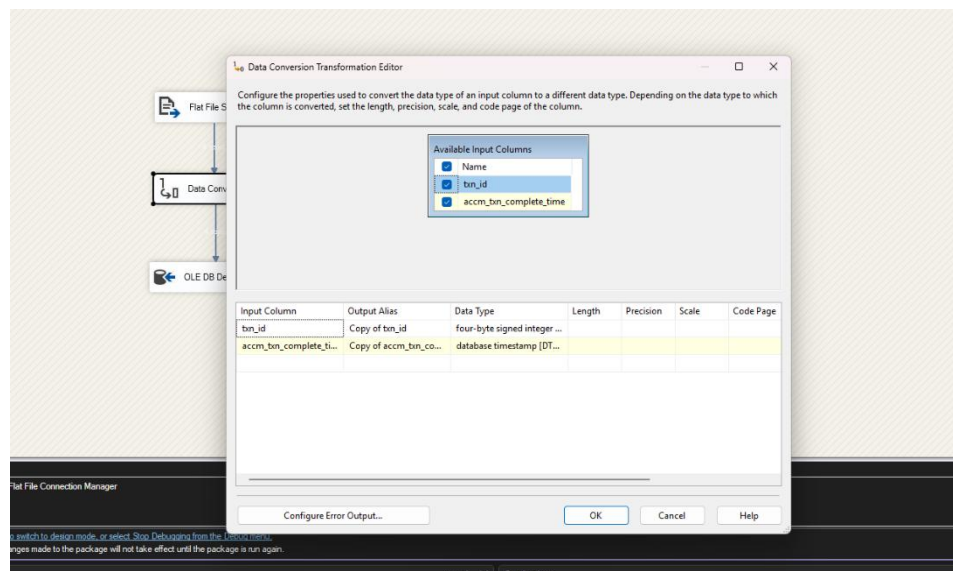


Figure 70: SSIS Data Conversion Transformation for Completion Update Records

6.6 Validation

Validation queries confirmed that the staging table loaded successfully, the matching fact rows were updated, process hours were calculated correctly, and unmatched transaction IDs were logged separately. The final results showed **4 rows** in the staging table, **3 completed transactions** updated in the fact table, and **1 unmatched transaction ID** recorded in the error table.

375 |
374 | SELECT TOP 20 *
375 | FROM DWBI_Staging.dbo.ETL_Error_FactCompletion_NoMatch
376 | ORDER BY LoadTimestamp DESC;
377 |
378 |

100 % ▾ × 70 ⚠ 0 ↑ ↓

Results Messages

	txn_id	accm_txn_complete_time	ErrorReason	LoadTimestamp
1	999999	2026-04-01 12:00:00.000	No matching IncidentID found in FactCrimeIncident	2026-03-30 02:17:55.047

Figure 71: Verification of Unmatched Transaction IDs Recorded in the Fact Completion ETL Error Table

The figure above shows the contents of **ETL_Error_FactCompletion_NoMatch** after package execution. The unmatched transaction ID **999999** was logged together with its completion timestamp and an error message stating that no matching **IncidentID** was found in **FactCrimeIncident**. This demonstrates the package's handling of missing transaction IDs.

Process-hours calculation validation

```

378 |
379 SELECT
380     IncidentID,
381     accm_txn_create_time,
382     accm_txn_complete_time,
383     txn_process_time_hours,
384     DATEDIFF(HOUR, accm_txn_create_time, accm_txn_complete_time) AS ExpectedHours
385 FROM DWBI_Warehouse.dbo.FactCrimeIncident
386 WHERE IncidentID IN (27602, 28028, 28089);

```

100 % ▾ 70 ⚠ 0 ⬆ ⬇

Results Messages

	IncidentID	accm_txn_create_time	accm_txn_complete_time	txn_process_time_hours	ExpectedHours
1	27602	2026-03-30 01:49:49.303	2026-03-31 10:00:00.000	33	33
2	28028	2026-03-30 01:49:49.303	2026-03-31 15:00:00.000	38	38
3	28089	2026-03-30 01:49:49.303	2026-04-01 09:30:00.000	56	56

Figure 72: Validation of Transaction Processing Hours Derived from Creation and Completion Timestamps

The figure shows the validation of the calculation of **txn_process_time_hours** by comparing the stored value with a recalculated **DATEDIFF(HOUR, accm_txn_create_time, accm_txn_complete_time)**. For all three updated rows, the stored value matches **ExpectedHours**, confirming that the calculation logic was implemented correctly.